

VERIQTA

40 REAL-WORLD DEVOPS INTERVIEW SCENARIOS

Real Problems.
Real Tools.
Real Production.
Real Answers.

Think
Troubleshoot
Solve

BUILD
TEST
DEPLOY
REPEAT

★
Real incidents
Real impact
Real learning

You Will Learn:

- ✓ How to think
- ✓ How to investigate
- ✓ How to solve
- ✓ How to prevent
- ✓ How to lead

Real interview questions. Real production issues.
Real troubleshooting. Real DevOps mindset.



 DOCKER	 KUBERNETES	 AWS	 TERRAFORM	 JENKINS
 LINUX	 GIT	 PROMETHEUS	 GRAFANA	 ANSIBLE
 HELM	 ARGO CD	 NGINX	 MYSQL	 ELK STACK

Perfect for:

- ✓ DevOps Engineers
- ✓ Site Reliability Engineers
- ✓ Cloud Engineers
- ✓ Platform Engineers
- ✓ Anyone who wants to level up

THIS HANDBOOK WILL HELP YOU:

- ✓ Crack DevOps interviews
- ✓ Solve real-world problems faster
- ✓ Think like a senior engineer
- ✓ Stand out from other candidates
- ✓ Build confidence in production situations

From Incidents
to Insights.
From Questions
to Expertise.

— VERIQTA —

1. KUBERNETES PODS ARE RUNNING, BUT THE APPLICATION IS UNREACHABLE

IMPORTANT !

Do not restart pods. Your goal is to find the real cause. Restarting hides the problem. Think. Isolate. Prove. Fix.

SCENARIO:

The deployment completed successfully. All pods show **Running**, but users cannot reach the application.

HIGH-LEVEL TRAFFIC FLOW



FAILURE CAN HAPPEN AT ANY POINT.

HOW TO ISOLATE THE FAILURE - STEP BY STEP

- 1. VERIFY PODS**
kubectl get pods -o wide
Are pods **Running** and **Ready**?
- 2. VERIFY SERVICE**
kubectl get svc -o wide
Check **ClusterIP**, **port**, **targetPort**.
- 3. CHECK ENDPOINTS**
kubectl get endpoints <svc-name> -o wide
Are endpoints present and pointing to pod IPs?
- 4. CHECK SELECTORS**
kubectl describe svc <svc-name>
Do selectors match pod labels?
- 5. CHECK READINESS PROBES**
kubectl describe pod <pod-name>
Look for failing readiness probes.
- 6. TEST INSIDE THE CLUSTER**
kubectl run -it --rm debug --image=curlimages/curl -- sh
• curl http://<service-name>:<port>
• curl http://<pod-ip>:<container-port>
- 7. CHECK INGRESS (IF USED)**
kubectl get ingress -o wide
kubectl describe ingress <ing-name>
Check rules, backend service, path, hosts.
- 8. CHECK NETWORKPOLICY**
kubectl get networkpolicy -A
kubectl describe networkpolicy <name>
Could it be blocking traffic?
- 9. CHECK DNS**
nslookup <hostname>
dig <hostname>
Does it resolve to the correct IP?
- 10. CHECK APPLICATION BINDING**
kubectl exec -it <pod-name> -- ss -tulpn | grep <port>
Is the app listening on 0.0.0.0 or only on localhost?
- 11. CHECK POD LOGS**
kubectl logs <pod-name> --tail=100
Look for errors, startup issues, binding problems.

ISOLATE → IDENTIFY → PROVE → FIX → VERIFY

INTERVIEW FOCUS:

- Service selectors
- Endpoints and EndpointSlices
- TargetPort versus containerPort
- Readiness probes
- Ingress routing
- NetworkPolicy
- DNS
- Application binding
- curl, kubectl describe, kubectl get endpoints, kubectl logs

COMMON ROOT CAUSES

- ⚠ Service selector does not match pod labels
- ⚠ Endpoints are empty
- ⚠ targetPort or containerPort mismatch
- ⚠ Readiness probe failing
- ⚠ Ingress misconfiguration
- ⚠ DNS resolving to wrong IP
- ⚠ NetworkPolicy blocking traffic
- ⚠ Application binds only to localhost



USEFUL COMMANDS CHEAT SHEET

PURPOSE	COMMAND
Check pods	kubectl get pods -o wide
Describe pod	kubectl describe pod <pod-name>
Check service	kubectl get svc -o wide
Describe service	kubectl describe svc <svc-name>
Check endpoints	kubectl get endpoints <svc-name> -o wide
Check endpoint slices	kubectl get endpointslice -l <label> kubernetes.io/service-name=<svc-name>
Check ingress	kubectl get ingress -o wide
Describe ingress	kubectl describe ingress <ing-name>
Check network policies	kubectl get networkpolicy -A
Describe network policy	kubectl describe networkpolicy <name>
DNS lookup	nslookup <hostname>
Port check inside pod	kubectl exec -it <pod-name> -- ss -tulpn
Pod logs	kubectl logs <pod-name> --tail=100
Test from inside cluster	kubectl run -it --rm debug --image=curlimages/curl -- sh

★ PRODUCTION NOTE

- ✓ Always follow the layers: DNS -> Ingress -> Service -> Endpoints -> Pod
- ✓ Prove each layer before moving to the next.
- ✓ Never assume the application is down.
- ✓ Logs and events tell the real story.
- ✓ Document your findings and fix the root cause.



VERIQTA

2. TERRAFORM WANTS TO RECREATE PRODUCTION INFRASTRUCTURE

SCENARIO:

A routine terraform plan suddenly shows that several production resources will be destroyed and recreated.

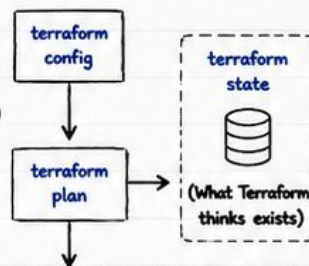


INTERVIEW FOCUS

- Terraform state
- ForceNew attributes
- Configuration drift
- Provider changes
- Resource addressing
- State inspection
- Lifecycle configuration
- Import and moved blocks
- Safe production change review

WHAT COULD BE HAPPENING?

- Resource has ForceNew attributes changed
- Configuration drift between state and real resource
- Provider version upgrade changed behavior
- Resource address changed (count → for_each, name change)
- Missing lifecycle rules
- Manually modified resources outside Terraform
- Module source or variables changed
- Dependency or implicit reference changed



≠

REAL INFRASTRUCTURE
(What actually exists)

COMMON ROOT CAUSES

1. ForceNew attribute changed (e.g. instance_type, name, engine_version)
2. Resource drifted from state
3. Provider upgrade changed default behavior
4. Resource address changed (count → for_each or name change)
5. Lifecycle rules are missing
6. Manual changes outside Terraform
7. Module or variable changes
8. Implicit dependency changed

Plan: 5 to add, 0 to change, 3 to destroy.

```
- aws_db_instance.prod_db      (destroy)
- aws_lb.app_alb               (destroy)
- aws_security_group.app_sg    (destroy)
+ aws_db_instance.prod_db      (create)
+ aws_lb.app_alb               (create)
+ aws_security_group.app_sg    (create)
```



DANGER!

Blindly applying this plan can destroy critical production resources and cause a major outage or data loss!

INVESTIGATION WORKFLOW



1. INSPECT PLAN CAREFULLY

terraform plan
-out=tfplan



2. INSPECT STATE vs REAL

terraform show
terraform state list



3. CHECK DRIFT & DIFFERENCES

terraform plan
-detailed-exitcode



4. IDENTIFY ROOT CAUSE

Compare config,
state & real infra



5. DECIDE SAFE ACTION

Fix config / state /
import / moved
blocks / lifecycle

USEFUL COMMANDS

- terraform plan -out=tfplan
- terraform show tfplan
- terraform state list
- terraform state show <resource_address>
- terraform state pull > state.json
- terraform refresh
- terraform providers
- terraform plan -detailed-exitcode
- terraform graph | dot -Tpng > graph.png

SAFE REMEDIATION OPTIONS

- ✓ Update configuration to match real infrastructure
- ✓ Use lifecycle { ignore_changes = [...] } for expected external changes
- ✓ Import existing resource: terraform import <addr> <id>
- ✓ Use moved blocks to change resource address safely (Terraform 1.1+)
- ✓ Adjust provider version carefully and test
- ✓ Re-run plan and validate before apply

PRODUCTION FAILURE

- Destroying a database = data loss
- Destroying an ALB = downtime
- Destroying a security group = lockout
- Destroying EBS = data loss
- Destroying IAM roles = permission failures
- Destroying VPC = total outage

THINK BEFORE YOU APPLY!



PRODUCTION NOTE

terraform apply is not a deployment command.
It is an infrastructure change command.
Always understand WHAT, WHY, and IMPACT before you apply.



SENIOR ENGINEER TIP

A good engineer explains the plan.
A senior engineer questions the plan.
A great engineer protects production.



YouTube
/veriqta



GitHub
/veriqta



LinkedIn
/veriqta



X
@veriqta



Instagram
@veriqta



Telegram
/veriqta

3. JENKINS PIPELINE SUDDENLY TAKES 45 MINUTES INSTEAD OF 10

SCENARIO:

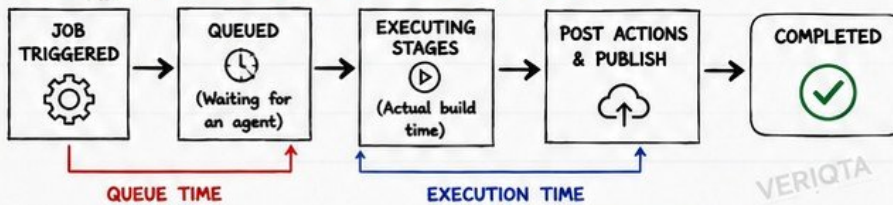
The pipeline still succeeds, but build duration has increased dramatically.



INTERVIEW FOCUS

- Jenkins agents
- Queue time vs execution time
- CPU and memory pressure
- Docker layer caching
- Dependency downloads
- Artifact repositories
- Network latency
- Parallel stages
- Workspace growth
- Pipeline profiling

1. UNDERSTAND WHERE THE TIME IS SPENT



Always split the total duration into Queue Time and Execution Time. Most slow pipelines are caused by one of these two.

3. COMMON ROOT CAUSES



AGENT CONTENTION:

Too many jobs, not enough executors, or long-running builds.



CPU / MEMORY PRESSURE:

Agent machine is overloaded → context switching → slow builds.



DOCKER LAYER CACHING LOST:

Cache not used → images rebuilt → takes much longer.



DEPENDENCY DOWNLOADS:

External repos slow or rate-limited. No caching (maven / npm / pip).



ARTIFACT REPOSITORIES:

Nexus / Artifactory / S3 / ECR slow or throttling.



NETWORK LATENCY:

High latency to Git, registries, package repos, or cloud services.



POOR PARALLELISM:

Stages running sequentially instead of in parallel.



WORKSPACE GROWTH:

Huge repos, many files, or not cleaning workspace.

2. QUICK DIAGNOSIS CHECKLIST

- ✓ Check queue time vs execution time
- ✓ Are agents busy or overloaded?
- ✓ High CPU / Memory on agents?
- ✓ Are builds I/O or network bound?
- ✓ Docker builds taking longer?
- ✓ Dependency downloads slow?
- ✓ Artifact repository / npm / maven slow?
- ✓ Network latency to external services?
- ✓ Too many sequential stages?
- ✓ Workspace size growing over time?
- ✓ Using parallel stages effectively?
- ✓ Any new steps added recently?
- ✓ Compare with last known good build

4. JENKINS ANALYSIS COMMANDS & PLUGINS

USEFUL PLUGINS

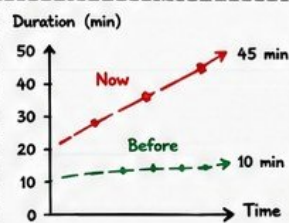
- Pipeline Stage View
- Build Timeline
- Analytics Plugin
- Resource Usage Plugin
- Disk Usage Plugin

USEFUL COMMANDS (ON AGENT)

- `top -b -n 1 | head -20`
- `free -h`
- `df -h`
- `iostat -xz 1 5`
- `du -sh * | sort -h`
- `docker system df`
- `docker image ls`

5. PIPELINE PROFILING APPROACH

- STEP 1:** Identify if queue time or execution time is the problem.
- STEP 2:** Check agent utilization (CPU, RAM, Disk, Network).
- STEP 3:** Review recent changes to Jenkinsfile or infrastructure.
- STEP 4:** Use Pipeline Stage View to find slow stages.
- STEP 5:** Investigate slow stage (logs, external calls, downloads).
- STEP 6:** Fix bottleneck, rerun, and compare metrics.



PRODUCTION FAILURE

- Longer build times delay releases.
- Slow feedback → more production bugs.
- Agents overloaded → other jobs blocked.
- Missed deployment windows.
- Developer productivity drops.
- Long pipelines hide real failures.

SLOW PIPELINES COST MONEY AND TRUST!



SENIOR ENGINEER TIP

- Always measure before you optimize.
- Small bottlenecks multiply across stages.
- Fast, reliable pipelines = high performing teams.

BEST PRACTICES

- ✓ Use enough, properly sized agents.
- ✓ Enable Docker layer caching.
- ✓ Cache dependencies (maven / npm / pip).
- ✓ Store artifacts close to agents.
- ✓ Use parallel stages where possible.
- ✓ Clean workspace and prune old data.

REMEMBER

A green build is good.
A fast green build is better.
A predictable fast green build is production ready.

4. AWS APPLICATION LOAD BALANCER RETURNS 503

SCENARIO:

EC2 INSTANCES ARE RUNNING AND APPEAR HEALTHY AT THE OPERATING SYSTEM LEVEL, BUT THE ALB RETURNS HTTP 503.



INTERVIEW FOCUS

- TARGET GROUPS
- REGISTERED TARGETS
- HEALTH CHECKS
- LISTENER RULES
- SECURITY GROUPS
- APPLICATION PORTS
- AVAILABILITY ZONES
- TARGET HEALTH REASON CODES
- ALB TROUBLESHOOTING

1. ALB REQUEST FLOW



2. COMMON 503 CAUSES

- 1 NO HEALTHY TARGETS IN TARGET GROUP
- 2 TARGETS FAILING HEALTH CHECKS
- 3 HEALTH CHECK PATH / PORT / PROTOCOL WRONG
- 4 APPLICATION NOT LISTENING ON EXPECTED PORT
- 5 SECURITY GROUP BLOCKING TRAFFIC
- 6 ALB IN ONE AZ BUT TARGETS ONLY IN ANOTHER
- 7 TARGET DRAINING DURING DEPLOYMENT
- 8 OVERLOADED TARGETS (TIMEOUTS)
- 9 STICKY SESSIONS MISCONFIGURED
- 10 LISTENER RULES FORWARDING TO WRONG TG

3. TARGET HEALTH REASON CODES (COMMON)

REASON CODE	WHAT IT MEANS
Target.ResponseCodeMismatch	Health check returned non-200 (or not the matcher)
Target.Timeout	Target did not respond in time
Target.ConnectionError	ALB could not connect to target (SG, NACL, app down)
Target.FailedHealthChecks	Target failed the configured number of checks
Target.NotRegistered	Target not registered in target group
Elb.InternalError	Internal ALB issue (rare)
Target.IpUnusable	IP not routable in the AZ
Target.DeregistrationInProgress	Target is draining connections

4. TROUBLESHOOTING CHECKLIST

- ✓ CHECK TARGET GROUP: ARE THERE ANY HEALTHY TARGETS?
- ✓ REVIEW HEALTH CHECK SETTINGS (PATH, PORT, PROTOCOL, TIMEOUTS).
- ✓ VERIFY SECURITY GROUP ALLOWS TRAFFIC FROM ALB SG.
- ✓ CONFIRM APPLICATION IS LISTENING ON EXPECTED PORT.
- ✓ CHECK TARGET HEALTH DETAILS AND REASON CODES.
- ✓ VERIFY LISTENER RULES FORWARD TO CORRECT TG.
- ✓ CONFIRM TARGETS ARE IN THE SAME AZ(S) AS ALB NODES.
- ✓ CHECK FOR RECENT DEPLOYMENTS OR CONFIG CHANGES.
- ✓ TEST FROM ALB TO TARGET: USE HEALTH CHECK PATH.
- ✓ USE ACCESS LOGS & CLOUDWATCH METRICS FOR CLUES.

5. USEFUL AWS CLI COMMANDS

- `aws elbv2 describe-target-health --target-group-arn <arn>`
Check target health + reason codes
- `aws elbv2 describe-target-groups --names <tg-name>`
View target group configuration
- `aws elbv2 describe-listeners --load-balancer-arn <arn>`
Check listener rules and ports
- `aws elbv2 describe-rules --listener-arn <arn>`
View rule conditions and forwarding
- `aws elbv2 describe-load-balancers --names <alb-name>`
Verify ALB scheme, AZs, SGs
- `aws elbv2 describe-tags --resource-arns <arn>`
Check naming, environment, ownership

6. KEY CLOUDWATCH METRICS

METRIC	WHAT IT SHOWS
HTTPCode_ELB_5XX_Count	5xx errors generated by ALB
HTTPCode_Target_5XX_Count	5xx errors from targets
TargetResponseTime	Latency between ALB and target
HealthyHostCount	Number of healthy targets
UnHealthyHostCount	Number of unhealthy targets
RequestCount	Total requests to ALB
RejectedConnectionCount	Connections rejected by ALB

PRODUCTION FAILURE

⚠️ BLINDLY RESTARTING EC2 INSTANCES WON'T FIX A 503 IF THE ROOT CAUSE IS IN:

- HEALTH CHECK CONFIG
- SECURITY GROUPS
- LISTENER RULES
- APPLICATION PORTS
- OR RESOURCE OVERLOAD

ALWAYS FIND THE **REAL CAUSE** BEFORE MAKING CHANGES!



SENIOR ENGINEER TIP

- ✓ A 503 FROM ALB = NO HEALTHY TARGETS. PERIOD.
- ✓ TRUST THE TARGET HEALTH REASON CODE. IT SAVES HOURS.
- ✓ AUTOMATE HEALTH CHECKS AND ALERTING.
- ✓ KEEP TARGET GROUPS SIMPLE AND ISOLATED BY SERVICE.
- ✓ VALIDATE CHANGES IN STAGING BEFORE PRODUCTION.



REMEMBER

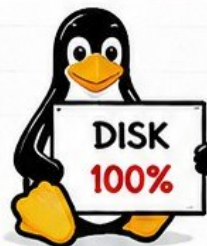
THE ALB IS ONLY AS GOOD AS THE HEALTHY TARGETS BEHIND IT. OBSERVE. VERIFY. THEN ACT.



5. LINUX SERVER DISK IS 100% FULL

SCENARIO:

A PRODUCTION LINUX SERVER
SUDDENLY REACHES 100%
DISK UTILIZATION.



INTERVIEW FOCUS

- df
- du
- lsof
- Deleted but open files
- Log growth
- inode exhaustion
- Docker storage
- journald
- Temporary files
- Safe cleanup

1. IMMEDIATE CHECKS

```
df -h      # Check disk usage by filesystem
df -i      # Check inode usage
uptime     # System load overview
free -h    # Memory overview
```



100%

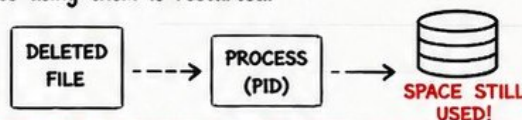
2. FIND WHAT IS USING SPACE

```
du -sh /*      # Top-level directories size
du -h --max-depth=1 /  # Find large directories
du -ahx /var | sort -hr | head -n 20  # Top 20 in /var
du -ahx / | sort -hr | head -n 20  # Top 20 in root (slow)
```

3. CHECK DELETED BUT OPEN FILES

```
lsof +L1      # List open files that were deleted
lsof +L1 | awk '{print $9, $7}' | sort -k2 -hr | head
# Biggest deleted files
```

These files still consume space until the process using them is restarted.



4. CHECK LOG GROWTH

```
du -sh /var/log/* | sort -hr  # Find large log files
ls -lh /var/log               # Inspect log directory
journalctl --disk-usage       # Size used by journald
journalctl --vacuum-size=1G   # Reduce journald size
tail -n 100 /var/log/messages # Check for log storms
```

5. CHECK INODE EXHAUSTION

```
df -i      # Check inode usage
find / -xdev -type f | wc -l  # Count files in filesystem
find /var -xdev -type f | wc -l  # Count files in /var
```

100% INODES
= Cannot create new files even if disk space is available.



6. DOCKER STORAGE USAGE

```
docker system df -h  # Docker disk usage summary
docker image prune -a  # Remove unused images
docker container prune  # Remove stopped containers
docker volume prune  # Remove unused volumes
docker builder prune  # Remove build cache
```



7. CHECK TEMPORARY FILES

```
du -sh /tmp/* 2>/dev/null  # Large files in /tmp
du -sh /var/tmp/* 2>/dev/null  # Large files in /var/tmp
find /tmp -type f -atime +7 -delete  # Clean old tmp files
find /var/tmp -type f -atime +7 -delete  # Clean old var tmp files
```



8. SAFE CLEANUP CHECKLIST (BEFORE DELETING ANYTHING)

- ✓ Confirm the filesystem that is 100% full.
- ✓ Understand what is consuming space (not just where).
- ✓ Verify if files are safe to delete (logs, cache, temp, old backups).
- ✓ Check if the file is still in use (lsof +L1).
- ✓ Ensure the application/service will not be affected.
- ✓ Take a snapshot or backup if unsure.
- ✓ Cleanup gradually and monitor after each step.
- ✓ Document what you changed.

COMMON SAFE CLEANUP TARGETS

- Old logs
- Old rotated logs
- Cache directories
- Temporary files
- Old packages (yum clean all / apt clean)
- Docker unused data
- Old backups (after verification)

PRODUCTION FAILURE

- Deleting active log files can hide the root cause.
- Removing files still in use won't free space.
- Deleting wrong data can cause outages.
- Cleaning without understanding = more downtime.



THINK FIRST. DELETE LAST. VERIFY ALWAYS.



SENIOR ENGINEER TIP

A good engineer never panics.
Investigate first. Free space second.
Prevent recurrence by fixing the root cause
(log rotation, monitoring, alerts).

REMEMBER

Disk full is almost never random.
It is usually logs, files, or processes.
Observe. Investigate. Clean. Prevent.



USEFUL MONITORING

- Disk usage alerts (70%, 80%, 90%).
- Inode usage alerts (80%, 90%).
- Log directory size alerts.
- Docker storage alerts.



6. KUBERNETES DEPLOYMENT WORKS, BUT THE NEW PODS KEEP RESTARTING

SCENARIO:

A NEW RELEASE DEPLOYS SUCCESSFULLY, BUT THE REPLACEMENT PODS ENTER CRASHLOOPBACKOFF.



CRASHLOOPBACKOFF
POD RESTARTING

INTERVIEW FOCUS

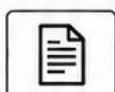
- kubectl logs
- Previous container logs
- kubectl describe
- Exit codes
- OOMKilled
- Environment variables
- Secrets
- ConfigMaps
- Startup dependencies
- Entrypoint failures
- Rollback decisions

1. TROUBLESHOOTING WORKFLOW



1. IDENTIFY
POD STATUS

kubectl get pods



2. DESCRIBE
THE POD

kubectl describe pod



3. CHECK
LOGS

kubectl logs



4. CHECK PREVIOUS
CONTAINER LOGS

kubectl logs --previous



5. ANALYZE
EXIT CODE

kubectl get pods -o wide



6. FIX &
REDEPLOY

Verify stability

2. IMPORTANT COMMANDS

```
kubectl get pods -n <ns>           # Check pod status
kubectl describe pod <pod> -n <ns> # Events, state, reason
kubectl logs <pod> -n <ns>         # Current container logs
kubectl logs <pod> -n <ns> --previous # Previous container logs
kubectl get pods -o wide -n <ns>    # Node, IP, status info
kubectl get events -n <ns> --sort-by=.lastTimestamp # Recent events
kubectl top pod <pod> -n <ns>       # CPU / Memory usage
kubectl exec -it <pod> -n <ns> -- sh # Debug inside container
kubectl get all -n <ns>             # Full resources overview
```

3. WHAT TO LOOK FOR IN 'KUBECTL DESCRIBE'

State:
Waiting / Running / Terminated

Reason:
CrashLoopBackOff

Last State:
Terminated

Exit Code:
(e.g., 1, 137, 139, 255)

Events:
Liveness probe failed, Back-off restarting failed container, OOMKilled



4. COMMON ROOT CAUSES

APPLICATION CRASH Bug, exception, wrong config, missing files, bad input.	ENTRYPOINT / COMMAND FAILURES Wrong command, missing script, bad permissions.	CONFIGMAP ISSUES Invalid data, missing keys, bad file mounts.
OOMKILLED (EXIT CODE 137) Memory limit too low or memory leak.	ENVIRONMENT VARIABLES Missing or incorrect env vars. Wrong values.	APPLICATION PORT / BINDING App not listening on expected port or address.
SECRETS / CONFIGS Missing or incorrect Secrets or ConfigMaps.	STARTUP DEPENDENCIES Database, API, or service not ready.	FILE SYSTEM / PERMISSIONS Read-only file system, permission denied.

5. COMMON EXIT CODES

EXIT CODE	MEANING	COMMON CAUSE
0	Success	Should not restart
1	General Error	Application error / misconfig
137	OOMKilled	Out of memory
139	Segmentation Fault	Memory access violation
143	Killed (SIGTERM)	Graceful shutdown
255	Unknown	Entrypoint / script failure

6. ROLLBACK DECISION GUIDE



ROLLBACK IMMEDIATELY IF:
User impact is high, root cause is unclear, data loss risk exists, or errors are widespread.



INVESTIGATE FIRST IF:
Issue is minor, reproducible, isolated to few pods, or rollback itself is risky.



SAFE ROLLBACK CHECKS:
Database migrations compatible, Secrets unchanged, traffic manageable.

ROLLBACK COMMANDS

```
kubectl rollout undo deployment/<name> -n <ns>
# Rollback to previous ReplicaSet

kubectl rollout history deployment/<name> -n <ns>
# View rollout history

kubectl rollout status deployment/<name> -n <ns>
# Monitor rollout
```



SENIOR ENGINEER TIP

- ✓ Never delete pods. Find the root cause.
- ✓ Always check previous logs – the real error is usually there.
- ✓ Understand the difference between infrastructure issues and application issues.
- ✓ Stabilize production first, then fix the root cause.



REMEMBER

CrashLoopBackOff is a symptom, not the root cause. Your job is to find WHY the application exits. Check → Analyze → Fix → Verify → Learn.

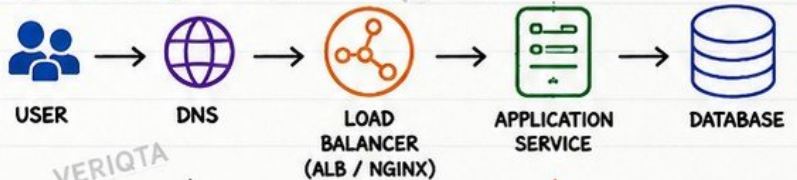
7. USERS REPORT INTERMITTENT 504 GATEWAY TIMEOUTS

SCENARIO

Most requests succeed, but traffic intermittently receives HTTP 504 responses during busy periods.



TYPICAL REQUEST FLOW



WHAT IS A 504 GATEWAY TIMEOUT?

A 504 means the proxy or gateway (waiting in front of your service) did not receive a response from the upstream server in time.

NOT always an application crash.

It is usually a latency or availability problem somewhere upstream.

INTERVIEW FOCUS

- Proxy timeout
- Upstream latency
- Application saturation
- Database latency
- Connection pools
- Load balancer metrics
- Network paths
- Distributed tracing
- Correlation between infrastructure and application metrics



COMMON ROOT CAUSES

- | | |
|------------------------------|--|
| ① PROXY / LB TIMEOUT TOO LOW | Timeout shorter than real response time under load. |
| ② UPSTREAM LATENCY SPIKES | Application or dependencies take longer during peak traffic. |
| ③ APPLICATION SATURATION | CPU, threads, or workers exhausted. |
| ④ DATABASE SLOW QUERIES | Locks, scans, or resource contention. |
| ⑤ CONNECTION POOL EXHAUSTED | No available connections to DB or downstream services. |
| ⑥ NETWORK ISSUES | Packet loss, latency, or intermittent routing problems. |
| ⑦ DEPENDENCY INSTABILITY | Third-party APIs or services slow or unreliable. |

KEY QUESTIONS TO ASK

- Does the 504 occur only during peak traffic?
- Which path returns the 504 (ALB, NGINX, API Gateway)?
- Is the upstream service responding slowly or not at all?
- Are there slow database queries or connection pool exhaustion?
- Are there any errors or timeouts in application logs?
- Are all AZs / instances experiencing the issue?

DANGER

- Blindly increasing timeouts hides the problem.
- Clients will wait longer and experience worse performance.
- Real root cause remains and may lead to full outage.

INVESTIGATION WORKFLOW



1. CONFIRM THE ISSUE

- Check user reports
- Reproduce if possible

2. CHECK GATEWAY / LB METRICS

- 5XX errors (504)
- Target response time
- Healthy vs unhealthy targets

3. CHECK APPLICATION METRICS

- Latency (p95, p99)
- Throughput
- Error rate
- Thread / worker usage

4. CHECK DEPENDENCIES

- Database performance
- Connection pool usage
- External API latency

5. TRACE & CORRELATE

- Use tracing (Jaeger / X-Ray)
- Find spans with high latency
- Correlate with metrics & logs

USEFUL COMMANDS

```

curl -I https://your-site.com           # Check response and status
curl -w "%{time_total}\n" -o /dev/null -s https://your-site.com # Total time
kubectl get pods -A                     # Check pod health
kubectl top pods -A                     # Check resource usage
kubectl logs <pod> -n <ns> --tail=200   # Recent logs
netstat -tulnp | grep <port>            # Check listening ports
ss -s                                    # Socket summary
dig your-site.com                       # DNS resolution check
traceroute your-site.com                 # Network path check
mysql -e "show processlist;"            # Check running DB queries
aws elbv2 describe-target-health --target-group-arn <arn> # ALB target health
aws elbv2 describe-load-balancers --names <name>          # LB details
jq . < file.json                         # Parse and inspect JSON logs
  
```

CORRELATE METRICS

- 504 spikes ↔ Upstream latency
- High CPU / memory ↔ Slow responses
- DB latency spike ↔ Application timeouts
- Connection pool maxed ↔ 504 errors
- Network packet loss ↔ Intermittent failures

TOOLS TO USE

- CloudWatch / Prometheus / Grafana
- ELB / NGINX Metrics
- Application APM (New Relic, AppDynamics)
- Distributed Tracing (Jaeger, AWS X-Ray)
- Logs (ELK / OpenSearch / Loki)

PRODUCTION NOTE



A 504 is the gateway telling you "I waited too long for upstream". Find the slowest dependency, fix performance, then tune timeouts last.



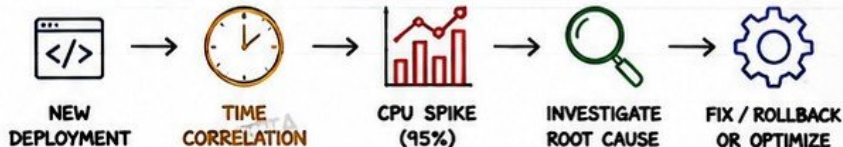
8. PRODUCTION CPU HITS 95% AFTER A DEPLOYMENT

SCENARIO

CPU utilization jumps immediately after a new release.



DEPLOYMENT CORRELATION



INTERVIEW FOCUS

- Deployment correlation
- Process-level investigation
- top and ps
- Container metrics
- Application profiling
- Traffic changes
- HPA behavior
- Rollback versus investigation
- Comparing old and new versions

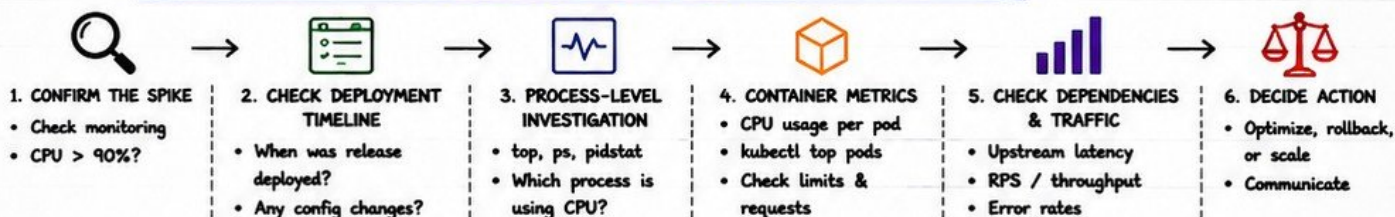
COMMON CAUSES OF CPU SPIKE AFTER DEPLOYMENT

①	Inefficient new code	Algorithm change, loops, heavy computations
②	Increased traffic	More users, bots, or load after release
③	Dependency slowness	Downstream service latency causing retries
④	Memory leak	Leads to GC pressure and high CPU
⑤	Logging explosion	Verbose logging or synchronous logging
⑥	Serialization / deserialization	Heavy JSON/XML processing
⑦	HPA not scaling	CPU high but replicas not increasing
⑧	Background jobs	New jobs, schedules, or workers added

SENIOR TIP

Always correlate the spike with the deployment time. Don't guess—prove it with metrics and logs.

INVESTIGATION WORKFLOW



USEFUL COMMANDS

Linux (Host Level)

```

top -p <pid> # CPU per process
ps -eo pid,ppid,cmd,%cpu,%mem --sort--%cpu | head -20
pidstat -u 1 5 # CPU usage per process
mpstat -P ALL 1 5 # CPU per core

```

Kubernetes

```

kubectl top nodes # Node CPU
kubectl top pods -A --sort-by=cpu
kubectl get pods -A -o wide
kubectl describe pod <pod> # Check limits/requests

```

Docker (if applicable)

```

docker stats # Live container metrics
docker top <container> # Processes in container

```

HPA BEHAVIOR CHECKLIST

- ☒ Is metrics-server running?
- ☒ Is CPU request set correctly?
- ☒ Is target CPU utilization realistic?
- ☒ Min / Max replicas configured?
- ☒ Any scaling cooldown delays?
- ☒ Is HPA status showing scaling?
- ☒ Check: `kubectl describe hpa <name>`

ROLLBACK VS INVESTIGATION

ROLLBACK WHEN:

- ✓ Critical impact to users
- ✓ Root cause unclear and risky
- ✓ New release is the likely cause
- ✓ Fast rollback is possible

INVESTIGATE WHEN:

- ✓ Impact is partial or acceptable
- ✓ Change is complex / high-risk
- ✓ Issue might be external
- ✓ Need data for long-term fix

COMPARE OLD VS NEW VERSION

ITEM	OLD VERSION	NEW VERSION	DIFFERENCE TO CHECK
CPU USAGE (per request)	Lower	Higher	Profiling / optimization needed
RESPONSE TIME	120ms	450ms	Check slow endpoints
DATABASE QUERIES	20 / req	150 / req	N+1 queries / inefficiency?
ERROR RATE	0.1%	1.2%	New bugs / exceptions?
DEPENDENCY LATENCY	50ms	200ms	Upstream issues?
MEMORY USAGE	512MB	900MB	Leaks or large objects?
CONNECTION POOL USAGE	30%	95%	Pool exhaustion?

PRODUCTION FAILURE



BLINDLY IGNORING THE CPU SPIKE CAN LEAD TO:

- Service saturation and slow responses
- Pod crashes due to resource exhaustion
- HPA unable to scale (wrong metrics)
- Cascading failures to dependencies
- Customer impact and revenue loss
- Emergency rollback under pressure

ALWAYS INVESTIGATE BEFORE YOU ESCALATE OR ROLLBACK!

9. GITHUB ACTIONS PIPELINE PASSES, BUT PRODUCTION DEPLOYMENT FAILS

SCENARIO

CI completes successfully.
The deployment stage reports success, but the new application version is not actually serving traffic.



PIPELINE APPEARS GREEN



CODE
PUSH



BUILD



TEST



DEPLOY



SUCCESS



BUT USERS
SEE OLD VERSION

INTERVIEW FOCUS

- Workflow conditions
- Deployment credentials
- Environment protection
- Kubernetes context
- Image tags
- Registry authentication
- Rollout verification
- False-positive pipelines
- Post-deployment validation



COMMON REASONS PIPELINE PASSES

①	WRONG KUBERNETES CONTEXT	Pipeline deploys to the wrong cluster or namespace.
②	OLD IMAGE DEPLOYED	Image tag is wrong, cached, or overwritten.
③	IMAGE NOT PULLED	K8s node pulls old cached image.
④	DEPLOYMENT NOT UPDATED	kubectl apply didn't trigger a rollout.
⑤	ROLLBACK BY AN OPERATOR	Argo CD / Flux / Operator reverted it.
⑥	SERVICE / INGRESS NOT UPDATED	Traffic still routed to old pods.
⑦	READINESS PROBE FAILING	New pods not ready, old pods still serve.
⑧	ENVIRONMENT PROTECTION	Approval skipped or sent to wrong env.
⑨	CONDITIONS SKIPPED STEPS	if: condition skipped deploy job.



SENIOR TIP

A green pipeline only means the steps ran. It does **NOT** guarantee the new version is serving traffic. Always verify in the real environment!

INVESTIGATION WORKFLOW



1. VERIFY DEPLOYMENT TARGET

- Check cluster context
- Check namespace
- kubectl config current-context

2. VERIFY IMAGE USED

- Check image tag
- kubectl get deploy -o yaml
- Check imagePullPolicy
- Check registry

3. VERIFY ROLLOUT STATUS

- kubectl rollout status deploy/APP
- kubectl get pods -l app=APP
- Check pod ages & versions

4. VERIFY TRAFFIC ROUTING

- kubectl get svc,ing -n ns
- Check selectors
- Check endpoints

5. VALIDATE REAL TRAFFIC

- curl the service
- Check logs & metrics
- Compare versions

USEFUL COMMANDS

```
kubectl config current-context
kubectl get deploy -A
kubectl rollout history deploy/APP
kubectl rollout undo deploy/APP
kubectl get pods -l app=APP -o wide
kubectl describe deploy/APP
kubectl get svc,ing -A
kubectl get endpoints -A
kubectl logs -l app=APP --tail=200
kubectl get events --sort-by='.lastTimestamp'
```

CHECKLIST BEFORE MARKING SUCCESS

- ✓ Correct cluster & namespace?
- ✓ Correct image tag deployed?
- ✓ New pods are running & ready?
- ✓ Service / Ingress points to new pods?
- ✓ Endpoints contain new pod IPs?
- ✓ Health checks return expected version?
- ✓ Real user traffic reaches the new version?
- ✓ Monitoring shows the new version?
- ✓ No automatic rollback detected?

FALSE-POSITIVE PIPELINES

- ✗ Deploy step ran but to wrong target
- ✗ kubectl apply returned success but nothing changed
- ✗ Image tag existed locally on nodes
- ✗ Old pods still matched the selector
- ✗ Readiness probe failed silently
- ✗ Traffic routed by external system
- ✗ Approval bypassed or wrong env
- ✗ No post-deploy verification

POST-DEPLOYMENT VALIDATION (MUST-HAVE)

- ✓ Automated smoke test
- ✓ API health checks
- ✓ Verify application version endpoint
- ✓ Metrics & logs validation
- ✓ Check key user flows
- ✓ Alert on version mismatch



PRODUCTION FAILURE

If the pipeline shows success but the old version serves traffic, users are impacted and engineers waste time. **Always verify** the deployment in the real environment before closing the change.

10. EC2 INSTANCE CANNOT ACCESS THE INTERNET FROM A PRIVATE SUBNET

SCENARIO

An application instance in a private subnet needs outbound access, but connections time out.

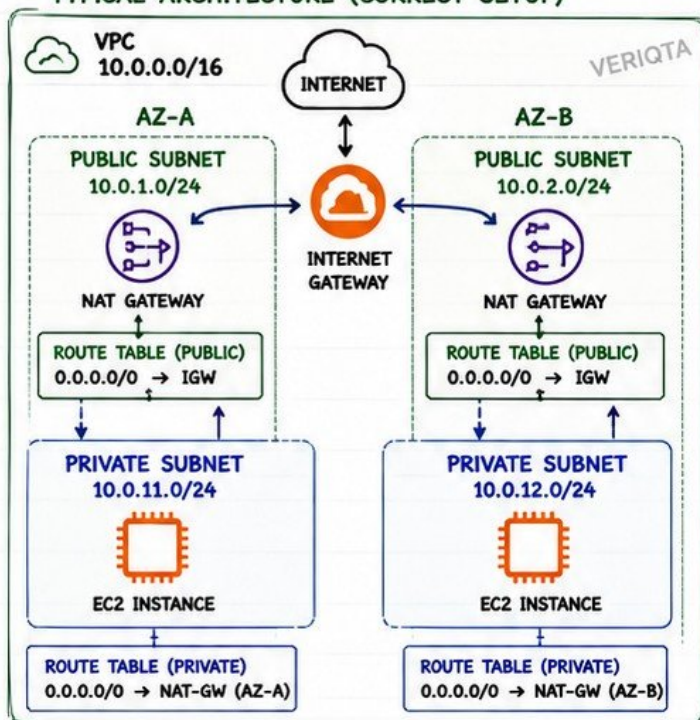


INTERVIEW FOCUS

- Route tables
- NAT Gateway
- Internet Gateway
- Public vs Private Subnet
- Security Groups
- NACLs
- DNS Resolution
- NAT Gateway Placement
- Return Traffic



TYPICAL ARCHITECTURE (CORRECT SETUP)



COMMON CAUSES & WHAT TO CHECK

#	POSSIBLE CAUSE	WHAT TO CHECK
1	Missing default route in private route table	- Route table for private subnet - Must have 0.0.0.0/0 → NAT-GW
2	NAT Gateway not present or in wrong AZ	- NAT Gateway exists? - In same AZ as private subnet?
3	NAT Gateway has no route to Internet	Public route table must have 0.0.0.0/0 → IGW
4	Internet Gateway missing or detached	- IGW attached to the VPC? - Route to IGW exists?
5	Security Group blocking outbound	- SG outbound rules allow traffic? - Default should allow all outbound
6	NACL blocking outbound or return traffic	- NACL outbound 1024-65535 ALLOW - NACL inbound 1024-65535 ALLOW
7	DNS resolution failure	- Check /etc/resolv.conf - Test: nslookup amazon.com
8	Return traffic blocked	- NACLs must allow return traffic - SGs are stateful (return allowed)
9	Endpoint or service issue	- Target service reachable? - Check service status / firewall

INVESTIGATION WORKFLOW



1. VERIFY SYMPTOM

- Try: curl https://amazon.com
- ping 8.8.8.8
- Note timeout behavior



2. CHECK ROUTE TABLE

- Private subnet route table
- Ensure 0.0.0.0/0 → NAT Gateway
- aws ec2 describe-route-tables



3. CHECK NAT GATEWAY

- NAT Gateway exists?
- In same AZ?
- Status = Available?
- aws ec2 describe-nat-gateways



4. CHECK IGW & PUBLIC RT

- IGW attached to VPC?
- Public route table has 0.0.0.0/0 → IGW?



5. CHECK SGs

- Outbound rule allows all or needed ports?
- SGs are stateful



6. CHECK NACLs

- Outbound: 1024-65535 ALLOW
- Inbound: 1024-65535 ALLOW



7. CHECK DNS

- cat /etc/resolv.conf
- nslookup example.com
- DNS must resolve



8. TEST AGAIN

- Re-test connectivity
- Check application logs / metrics

USEFUL COMMANDS

```
aws ec2 describe-route-tables # Check route tables
aws ec2 describe-nat-gateways # Check NAT Gateway
aws ec2 describe-internet-gateways # Check IGW
aws ec2 describe-security-groups # Check SG rules
aws ec2 describe-network-acls # Check NACL rules
nslookup amazon.com # Test DNS
curl -I https://amazon.com # Test outbound access
traceroute 8.8.8.8 # Trace network path
```

KEY NOTES

- Private subnet resources need a route to a NAT Gateway for outbound internet access.
- NAT Gateway must be in the same AZ as the private subnet for best performance and to avoid cross-AZ charges.
- IGW is required for internet access and must be attached to the VPC.
- NACLs are stateless - return traffic must be explicitly allowed.
- Security Groups are stateful - outbound rule is usually enough for return traffic.
- Always verify DNS - most application errors start here.

PRODUCTION DANGER

Deploying without proper egress paths can cause:

- Application failures
- Package updates fail
- External API timeouts
- Customer-facing outages
- Data sync or backups fail

VERIFY BEFORE YOU DEPLOY!



11. KUBERNETES SERVICE WORKS INTERNALLY BUT NOT THROUGH INGRESS

SCENARIO: The service responds correctly when accessed from inside the cluster, but the public hostname fails.

WHAT THIS REALLY MEANS

- The Service and Pods are healthy.
- Ingress or external traffic path is broken.
- Problem is between the Ingress controller and the outside world.

★ INTERVIEW FOCUS

- 1 IngressClass
- 2 Ingress controller
- 3 Host rules
- 4 Paths
- 5 TLS
- 6 Backend service
- 7 DNS
- 8 Load balancer
- 9 Controller logs
- 10 Certificate configuration

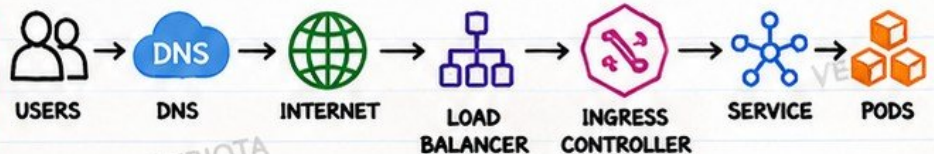
TROUBLESHOOTING CHECKLIST

- 1 DNS resolves to correct IP
- 2 Load balancer is healthy
- 3 Ingress controller pods are running
- 4 IngressClass is correct and active
- 5 Ingress resource exists and is valid
- 6 Host rule matches the request
- 7 Path rule matches the request
- 8 TLS certificate is valid and not expired
- 9 Backend service has endpoints
- 10 Controller logs show no errors

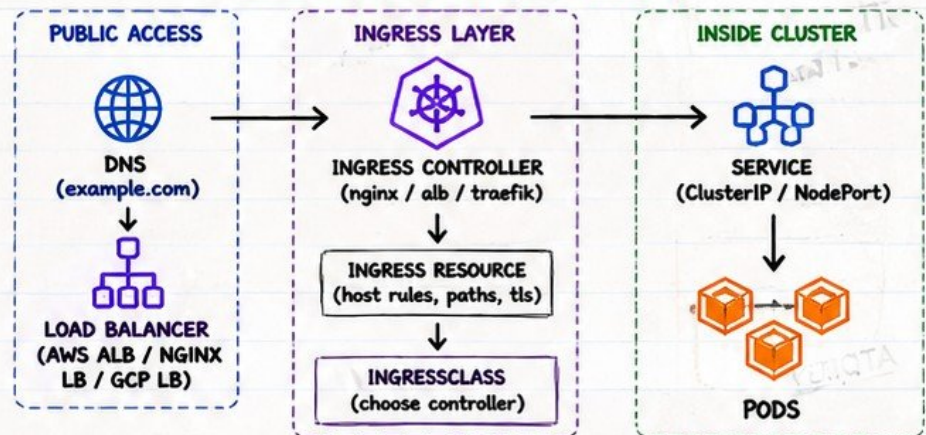
⚠ COMMON ROOT CAUSES

- DNS pointing to wrong load balancer
- IngressClass does not match controller
- Host rule mismatch
- Path rule not matching (Prefix vs Exact)
- TLS certificate missing or invalid
- Backend service has no endpoints
- Security groups / firewall blocking traffic
- Ingress controller configuration error

HIGH LEVEL TRAFFIC FLOW



INGRESS ARCHITECTURE (EXAMPLE)



COMMANDS TO INVESTIGATE

WHAT TO CHECK	COMMAND
Ingress resources	<code>kubectl get ingress -A</code>
Ingress details	<code>kubectl describe ingress <name> -n <ns></code>
IngressClass	<code>kubectl get ingressclass</code>
Ingress controller pods	<code>kubectl get pods -n <ingress-ns></code>
Controller logs	<code>kubectl logs -n <ingress-ns> -l app.kubernetes.io/name=ingress-nginx</code>
Service endpoints	<code>kubectl get endpoints <svc-name> -n <ns></code>
DNS lookup	<code>dig +short example.com</code>
Test from inside cluster	<code>kubectl run curl --image=curlimages/curl -it --rm --sh -c "curl -v http://<service>.<ns>.svc.cluster.local"</code>
Test via Ingress	<code>curl -k -I https://example.com</code>
Check events	<code>kubectl get events -n <ns> --sort-by=.lastTimestamp</code>

💡 REAL WORLD TIP

Most Ingress issues are not in Kubernetes Services or Pods. They are in DNS, Load Balancer, Ingress rules, or TLS. Always verify the entire path.

★ PRODUCTION NOTE

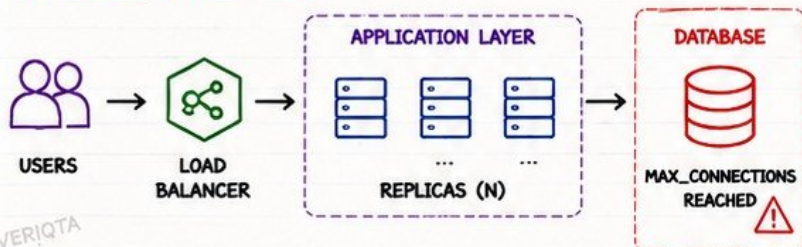
Always monitor:

- Ingress controller metrics
- 4xx/5xx rates
- TLS certificate expiration
- Backend service health

12. PRODUCTION DATABASE CONNECTIONS ARE EXHAUSTED

SCENARIO: The application starts returning errors because the database has reached its connection limit.

TYPICAL ARCHITECTURE



INTERVIEW FOCUS

- ✓ Connection pools
- ✓ Leaked connections
- ✓ Application replicas
- ✓ Database limits
- ✓ Long-running queries
- ✓ Idle sessions
- ✓ Scaling
- ✓ Pool sizing
- ✓ PgBouncer or equivalent
- ✓ Why increasing max_connections may not solve the problem

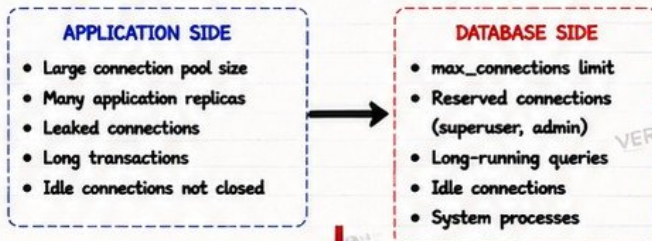
★ REAL WORLD IMPACT

- New requests fail with "Too many connections"
- Existing requests may slow down or time out
- Connection pool threads get blocked
- Application errors spike
- Users experience timeouts and failures

COMMON SYMPTOMS

SYMPTOM	WHAT YOU SEE
✗ Application errors	Too many connections / connection refused
🕒 High waiting time	Requests queue up waiting for a free connection
📉 Slow queries	Database starts rejecting or delaying new work
📊 DB metrics	Active connections at or near max limit
👤 User impact	Time outs, failed logins, partial outages

WHERE CONNECTIONS CAN GET CONSUMED



⚠️ CONNECTION POOL EXHAUSTED
NEW CONNECTIONS ARE REJECTED

COMMANDS TO INVESTIGATE (POSTGRESQL EXAMPLE)

WHAT TO CHECK	COMMAND
Current connections	<code>SELECT count(*) FROM pg_stat_activity;</code>
Connection details	<code>SELECT * FROM pg_stat_activity ORDER BY state, query_start;</code>
Long running queries	<code>SELECT pid, username, state, query, now()-query_start as duration FROM pg_stat_activity WHERE state = 'active' ORDER BY duration DESC;</code>
Idle in transaction	<code>SELECT pid, username, state, query, now()-query_start FROM pg_stat_activity WHERE state = 'idle in transaction';</code>
Max connections	<code>SHOW max_connections;</code>
Database stats	<code>SHOW superuser_reserved_connections;</code>
Blocked queries	<code>SELECT * FROM pg_locks;</code>
Who is connected	<code>SELECT * FROM pg_stat_activity;</code>

WHY INCREASING max_connections MAY NOT FIX IT

- ✗ More connections = more memory per connection.
- ✗ Database memory gets exhausted.
- ✗ Context switching increases → performance drops.
- ✗ Long running queries still hold connections.
- ✗ Leaked connections will still leak.
- ✗ System becomes unstable under high connection count.
- ✗ Does not address application connection management.

FIX THE ROOT CAUSE, NOT JUST THE LIMIT.

TROUBLESHOOTING CHECKLIST

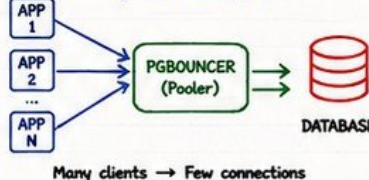
- ① Confirm the error and check current connection count.
- ② Identify which applications/replicas are consuming connections.
- ③ Check connection pool configuration in the application.
- ④ Look for leaked connections in code or ORM.
- ⑤ Identify long-running or idle queries.
- ⑥ Check for "idle in transaction" sessions.
- ⑦ Review database max_connections and reserved connections.
- ⑧ Consider PgBouncer or connection pooling layer.
- ⑨ Implement fixes and monitor before scaling limits.

SOLUTION OPTIONS

1. OPTIMIZE APPLICATION

- Right-size connection pool
- Close connections properly
- Fix connection leaks
- Reduce long transactions
- Use prepared statements

2. USE CONNECTION POOLER (PgBouncer Example)



3. SCALE DATABASE

- Increase instance size
- Add read replicas
- Partition workload
- Improve queries
- Monitor and tune

PRODUCTION NOTE

💡 Connection exhaustion is almost always an application or design issue, not just a database configuration issue.

REMEMBER

🎯 Healthy connection management = Stable database + Happy users.

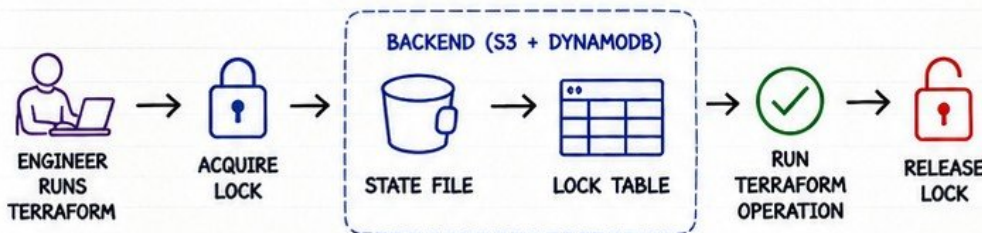
13. TERRAFORM STATE IS LOCKED DURING AN EMERGENCY DEPLOYMENT

SCENARIO: An engineer needs to deploy an urgent infrastructure change, but Terraform reports that the remote state is locked.

INTERVIEW FOCUS

- State locking
- Concurrent operations
- Backend behavior
- Identifying the lock owner
- Failed Terraform runs
- force-unlock
- Risks of manually removing locks
- Safe recovery

HOW TERRAFORM STATE LOCKING WORKS



WHY LOCKS EXIST

- ✓ Prevents concurrent writes to state
- ✓ Protects from state corruption
- ✓ Ensures infrastructure consistency
- ✓ Avoids race conditions

WHAT THE ERROR LOOKS LIKE

```

Error: Error acquiring the state lock
Error message: ConditionalCheckFailedException:
The conditional request failed
Lock Info:
ID:          9d5f4c3a-8c2e-4d7d-9b2a-8e7f9c6a1234
Path:        s3://my-terraform-state/prod/terraform.tfstate
Operation:   OperationTypeApply
Who:         engineer@company.com
Version:     1.7.5
Created:     2025-05-12 10:45:32 UTC
Info:
  
```

IDENTIFY THE LOCK OWNER

CHECK DYNAMODB LOCK TABLE

```

aws dynamodb get-item \
  --table-name terraform-locks \
  --key '{"LockID":{"S":"prod/terraform.tfstate"}}'
  
```

Look for:

- Who (user)
- Operation (apply/plan/destroy)
- Created (timestamp)
- Info (Terraform version)

COMMON REASONS LOCK EXISTS

- 🕒 Terraform process is still running
- ❌ Previous run crashed or was interrupted
- 📶 Network issue during operation
- 👤 Another engineer is working
- ⚙️ State lock from failed backend cleanup

SAFE RECOVERY PROCESS (STEP BY STEP)

- 1 Investigate the lock (do not unlock blindly).
- 2 Identify who owns the lock and what operation it was doing.
- 3 Verify if the Terraform process is still running.
- 4 Contact the owner if possible.
- 5 Confirm it is safe to unlock.
- 6 Use force-unlock with the Lock ID.
- 7 Run terraform plan to verify state is healthy.
- 8 Proceed with the emergency deployment.

FORCE UNLOCK COMMAND

```

terraform force-unlock 9d5f4c3a-8c2e-4d7d-9b2a-8e7f9c6a1234
Example Output:
Released state lock. This will allow local state changes to be made.
  
```

⚠️ RISKS OF MANUAL / FORCE UNLOCK

- Another operation may still be running.
- State corruption is possible.
- Partial writes can lead to drift.
- Resources may be created twice or destroyed.
- Can break remote state consistency.

ONLY UNLOCK WHEN YOU ARE 100% SURE.

FAILED TERRAFORM RUNS

If Terraform crashes or exits unexpectedly:

- Lock may remain in backend.
- Check CI/CD logs.
- Check system processes.
- Wait for lock to expire (if backend supports TTL).

Common in:

- CI/CD pipeline failures
- Network timeouts
- Agent crashes

BACKEND BEHAVIOR

BACKEND	LOCKING MECHANISM	NOTES
S3 + DynamoDB	DynamoDB conditional write	Recommended for production
Azure Storage	Blob lease	Lease breaks after timeout
GCS	Object generation / lock file	Uses GCS object locking
Terraform Cloud	Built-in locking	Managed automatically
Local State	No locking	Do not use in team environments

PRODUCTION NOTE

- ✓ Always use remote state with locking.
- ✓ Never disable locking in production.
- ✓ Educate the team on lock etiquette.
- ✓ Automate notifications for stale locks.
- ✓ Document the recovery procedure.
- ✓ State is your source of truth.

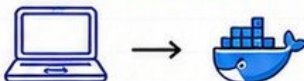
PROTECT STATE, PROTECT PRODUCTION.

14. DOCKER CONTAINER WORKS LOCALLY BUT FAILS IN PRODUCTION

SCENARIO: The exact image appears to work on a developer laptop but fails when deployed in production.

WHY IT WORKS LOCALLY BUT FAILS IN PRODUCTION

LOCAL ENVIRONMENT



- Different OS / architecture
- Extra environment variables
- Local files and mounts
- Network access is open
- DNS resolves differently
- Higher resources available
- Running as root
- Secrets in .env files

≠

PRODUCTION ENVIRONMENT



- Minimal base OS
- Strict environment variables
- No local files
- Network policies / firewalls
- DNS / service discovery
- Resource limits enforced
- Runs as non-root user
- Secrets from secure store

INTERVIEW FOCUS

- ✓ Environment differences
- ✓ Architecture differences
- ✓ Environment variables
- ✓ Volumes
- ✓ File permissions
- ✓ Networking
- ✓ DNS
- ✓ Resource limits
- ✓ Secrets
- ✓ Container user
- ✓ Runtime configuration

REAL WORLD IMPACT

- App crashes on startup
- Cannot connect to database
- Permission denied errors
- File not found errors
- Timeouts / network failures
- High CPU / OOMKilled
- Intermittent production issues

COMMON ROOT CAUSES AND WHAT TO CHECK

AREA	WHAT CAN GO WRONG	WHAT TO CHECK	EXAMPLE COMMANDS / ACTIONS
ENVIRONMENT DIFFERENCES	Different OS, packages, kernel, or architecture.	OS version, libc, timezone, locale, installed tools.	<code>uname -a</code> <code>cat /etc/os-release</code> <code>ldd --version</code>
ARCHITECTURE DIFFERENCES	Image built for amd64 but running on arm64 (or vice versa).	Container and host architecture compatibility.	<code>docker inspect --format '{{.Os}}/{{.Architecture}}' <image></code> <code>docker run --rm <image> uname -m</code>
ENVIRONMENT VARIABLES	Missing or different env vars change application behavior.	Compare all required environment variables.	<code>docker run --rm -e VAR=value <image> env sort</code> <code>kubectl set env --list deployment/<name></code>
VOLUMES AND FILE SYSTEM	Files that exist locally are not present in prod. Mount paths differ.	Volume mounts, read-only filesystems, missing paths.	<code>docker run --rm -v /host/path:/app/path <image> ls -l /app/path</code> <code>kubectl describe pod <pod> grep -A 5 Volumes</code>
FILE PERMISSIONS AND OWNERSHIP	Running as non-root user cannot write or read required files.	File ownership, group, mode, read/write access.	<code>docker run --rm <image> ls -la /app</code> <code>kubectl exec -it <pod> -- ls -la /app</code> <code>id</code>
NETWORKING	Firewalls, NetworkPolicy or closed ports block connections.	Service endpoints, policies, port exposure.	<code>docker run --rm <image> curl -v http://service:8080/health</code> <code>kubectl get networkpolicies -A</code>
DNS	Service names or external hosts do not resolve.	DNS config, search domains, CoreDNS / VPC DNS.	<code>docker run --rm <image> nslookup service</code> <code>cat /etc/resolv.conf</code>
RESOURCE LIMITS	Low memory/CPU causes slowdowns or OOMKilled.	Limits vs requests, actual usage in production.	<code>docker stats <container></code> <code>kubectl top pod <pod></code> <code>kubectl describe pod <pod> grep -i oom</code>
SECRETS	Secrets not available or incorrect in production.	Secret names, mounts, permissions, formats.	<code>kubectl describe secret <name></code> <code>kubectl logs <pod> grep -i secret</code>
CONTAINER USER	Runs as different user with fewer privileges.	USER in Dockerfile, runAsUser in K8s.	<code>docker run --rm <image> whoami</code> <code>kubectl exec -it <pod> -- id</code>
RUNTIME CONFIGURATION	Different command, args, healthcheck or entrypoint.	CMD, ENTRYPOINT, probes, resources, restart policy.	<code>docker inspect <container> grep -A 10 "Config"</code> <code>kubectl describe pod <pod> grep -A 20 "Liveness\ Readiness"</code>

TROUBLESHOOTING CHECKLIST

- 1 Reproduce the issue in a similar environment.
- 2 Check logs first: container, app, system.
- 3 Compare environment variables.
- 4 Verify network connectivity and DNS.
- 5 Check file permissions and volumes.
- 6 Validate secrets and configuration files.
- 7 Check resource usage and limits.
- 8 Test with minimal image if needed.



PRODUCTION LESSON

- What works on your laptop is not the same as production.
- Make environments as similar as possible.
 - Use infrastructure as code and version control.
 - Avoid hidden dependencies on your machine.
 - Always test with realistic resources and users.
 - Observability (logs, metrics, traces) is key.

CONSISTENCY IS THE FOUNDATION OF RELIABLE AND PREDICTABLE CONTAINERS.

QUICK DEBUG COMMANDS

```
docker logs <container> --tail 200
docker exec -it <container> sh
kubectl logs <pod> --previous
kubectl describe pod <pod>
kubectl exec -it <pod> -- sh
kubectl get events --sort-by=.metadata.creationTimestamp
curl -v <service>:<port>/health
env | sort
```

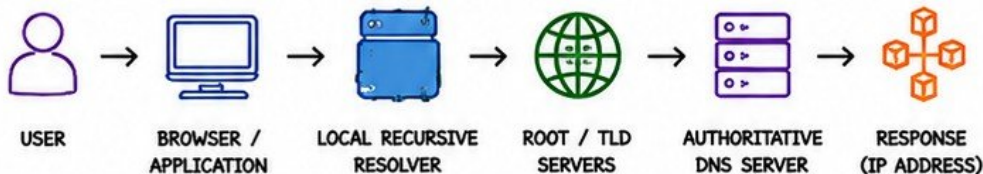
15. DNS WORKS FOR SOME USERS BUT NOT OTHERS

SCENARIO: A new application endpoint was released. Some users reach the new environment while others still reach the old one.

INTERVIEW FOCUS

- ✓ DNS TTL
- ✓ Recursive resolvers
- ✓ Client caching
- ✓ Route 53 records
- ✓ Alias records
- ✓ CDN caching
- ✓ DNS propagation misconceptions
- ✓ dig and nslookup
- ✓ Safe DNS migrations

HOW DNS RESOLUTION WORKS



REAL WORLD IMPACT

- Users hit different environments
- Session inconsistencies
- Login / payment failures
- Data mismatches
- Hard to reproduce issues
- Loss of trust and revenue

WHY SOME USERS SEE OLD, OTHERS NEW

- High TTL values cause long caching.
- Different recursive resolvers have different cache states.
- Client / OS / browser caching.
- CDN / edge caching returns old content.
- Some DNS records updated, others not.
- ISP DNS servers update at different times.
- Misunderstanding of DNS propagation.

KEY DNS CONCEPTS

CONCEPT	EXPLANATION
TTL (Time To Live)	How long DNS records are cached.
Recursive Resolver	DNS server that caches responses for clients.
Client Caching	OS / browser may cache DNS results.
Route 53 Records	Manage DNS in AWS Route 53.
Alias Records	AWS Route 53 alias points to AWS resources (ALB, CloudFront, S3).
CDN Caching	Edge locations may cache old content even after DNS update.
Propagation	Not instant. Each resolver updates at different times.

COMMON MISCONCEPTIONS

- ✗ DNS changes are instant.
- ✗ Clearing local cache fixes everyone.
- ✗ Changing TTL after the fact helps immediately.
- ✗ All DNS servers update at the same time.
- ✗ Lowering TTL right before a change always prevents issues.

USEFUL COMMANDS

PURPOSE	DIG COMMAND EXAMPLE	NSLOOKUP EXAMPLE	WHAT TO CHECK
Check A record	<code>dig example.com A +noall +answer</code>	<code>nslookup example.com</code>	Returned IP address
Check TTL	<code>dig example.com A +noall +answer</code>	<code>nslookup -query=ttl example.com</code>	TTL value
Use specific DNS server	<code>dig @8.8.8.8 example.com</code>	<code>nslookup example.com 8.8.8.8</code>	Resolver differences
Check all record types	<code>dig example.com ANY +noall +answer</code>	<code>nslookup -type=any example.com</code>	Record types present
Trace resolution path	<code>dig example.com +trace</code>	<code>nslookup -type=ns example.com</code>	Where resolution goes
Check AAAA (IPv6)	<code>dig example.com AAAA +noall +answer</code>	<code>nslookup -type=aaaa example.com</code>	IPv6 records
Check MX record	<code>dig example.com MX +noall +answer</code>	<code>nslookup -type=mx example.com</code>	Mail routing

ROUTE 53 RECORD TYPES YOU WILL USE

TYPE	PURPOSE
A	Maps domain to IPv4 address.
AAAA	Maps domain to IPv6 address.
CNAME	Points domain (not root) to another domain.
ALIAS	Points to AWS resource (ALB, CloudFront, S3, etc.).
MX	Mail server records.
TXT	Verification, SPF, DKIM, misc.

SAFE DNS MIGRATION STRATEGIES

- ① Lower TTL (24-48h before change).
- ② Verify all records and targets.
- ③ Deploy new environment and test.
- ④ Update DNS records.
- ⑤ Monitor from multiple regions / networks.
- ⑥ Keep old environment ready for rollback.
- ⑦ Increase TTL back after stability.

TROUBLESHOOTING CHECKLIST

- ① Check DNS record in Route 53.
- ② Verify TTL values.
- ③ Test with different DNS servers.
- ④ Check client / OS cache.
- ⑤ Test from another network / ISP.
- ⑥ Check CDN / edge cache.
- ⑦ Use dig +trace to see full path.
- ⑧ Wait and monitor propagation.



IMPORTANT NOTE

DNS is a distributed system. Different resolvers cache at different times. Always test from multiple locations and networks before and after changes.

16. KUBERNETES NODE BECOMES NOTREADY DURING PEAK TRAFFIC

SCENARIO

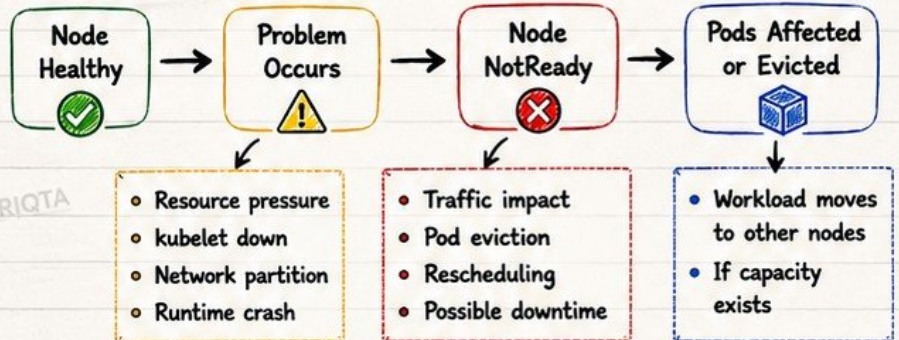
One worker node becomes NotReady while production traffic is high.

INTERVIEW FOCUS

- ✓ Node conditions
- ✓ kubelet
- ✓ MemoryPressure
- ✓ DiskPressure
- ✓ PIDPressure
- ✓ Network problems
- ✓ Container runtime
- ✓ Pod eviction
- ✓ cordon and drain
- ✓ Workload redistribution



WHAT HAPPENS WHEN A NODE GOES NOTREADY?



1. CHECK NODE CONDITIONS

```
kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
node-01	NotReady	<none>	12d	1.27.8

```
kubectl describe node node-01 | grep -A2 Conditions
```

TYPE	STATUS	LAST HEARTBEAT	LAST TRANSITION
MemoryPressure	False	2m	10m
DiskPressure	True	2m	2m
PIDPressure	False	2m	12m
Ready	False	2m	2m

3. INVESTIGATION COMMANDS

```

kubectl describe node node-01 # Details and events
kubectl get pods -o wide     # See affected pods
ssh node-01                  # Login to the node
systemctl status kubelet     # Check kubelet
journalctl -u kubelet -xe    # Kubelet logs
df -h                         # Disk usage
free -h                      # Memory usage
ps -eo pid,ppid,cmd | wc -l   # Check PID usage
crictl info                  # Container runtime status
ping <apiserver-ip>          # Network connectivity
dmesg | tail -n 50           # Kernel / OOM / disk errors
  
```

2. COMMON REASONS



RESOURCE PRESSURE

CPU, Memory, Disk or PID exhaustion causes kubelet to report NotReady.



KUBELET DOWN

kubelet service stopped or crashed.



CONTAINER RUNTIME FAILURE

containerd or cri-o is down.



NETWORK PROBLEMS

Node cannot communicate with the API server.



DISK ISSUES

Disk full, inode exhaustion, slow I/O.

4. REMEDIATION STEPS

- 1 Identify the root cause using the investigation commands.
- 2 Fix the underlying issue (disk, memory, kubelet, network, etc).
- 3 If node is unstable, CORDON it:

```
kubectl cordon node-01
```

- 4 If workloads must be moved, DRAIN it safely:

```
kubectl drain node-01 --ignore-daemonsets --delete-emptydir-data
```

- 5 Once fixed, UNCordon:

```
kubectl uncordon node-01
```

- 6 Verify node is Ready:

```
kubectl get nodes
```



PRODUCTION NOTE

- A single NotReady node can cause replica shortages.
- Evictions can increase latency during peak traffic.
- Always design clusters with enough spare capacity.
- Use PodDisruptionBudgets to protect critical workloads.



SENIOR ENGINEER TIP

Do not rush to reboot the node. Understand WHY it went NotReady first. If you drain blindly, you can make the outage worse.

BEST PRACTICES

- Monitor node conditions.
- Set alerts for DiskPressure, MemoryPressure, PIDPressure.
- Keep kubelet and runtime healthy.
- Regularly review node resources.

17. PROMETHEUS STOPS COLLECTING METRICS FROM ONE SERVICE

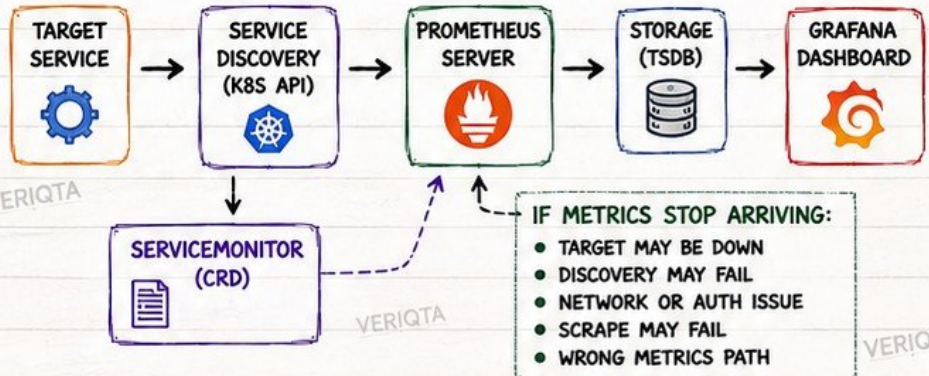
SCENARIO
DASHBOARDS SUDDENLY SHOW MISSING DATA FOR ONE APPLICATION WHILE OTHER SERVICES CONTINUE REPORTING NORMALLY.

INTERVIEW FOCUS

- ✓ PROMETHEUS TARGETS
- ✓ SERVICE DISCOVERY
- ✓ LABELS
- ✓ SERVICEMONITOR
- ✓ ENDPOINTS
- ✓ NETWORK ACCESS
- ✓ METRICS PATH
- ✓ AUTHENTICATION
- ✓ SCRAPE ERRORS
- ✓ CARDINALITY CONSIDERATIONS



HOW PROMETHEUS COLLECTS METRICS



1. CHECK PROMETHEUS TARGETS

```
curl -s http://prometheus:9090/api/v1/targets | jq .
```

JOB	INSTANCE	STATE	LAST SCRAPE	ERROR
api-service	10.244.1.15:8080	UP	5s ago	-
auth-service	10.244.2.11:8080	UP	4s ago	-
billing-service	10.244.3.9:8080	DOWN	-	context deadline exceeded
payment-service	10.244.4.7:8080	UP	6s ago	-

- IF STATE IS DOWN, PROMETHEUS IS NOT ABLE TO SCRAPE THE METRICS.

2. VERIFY SERVICEMONITOR

```
kubectl get servicemonitor -A
```

NAMESPACE	NAME	SELECTOR	ENDPOINTS	AGE
monitoring	billing-sm	app=billing	0/1	5d

- ENDPOINTS 0/1 MEANS PROMETHEUS FOUND THE SERVICE BUT NO ENDPOINT IS AVAILABLE.

3. CHECK ENDPOINTS FOR THE SERVICE

```
kubectl get endpoints billing-service -n production
```

NAME	ENDPOINTS	AGE
billing-service	<none>	5d

NO ENDPOINTS = PODS NOT READY OR SELECTORS DO NOT MATCH.

PRODUCTION NOTE

- A SINGLE SERVICE LOSING METRICS IS OFTEN NOT AN INFRASTRUCTURE PROBLEM.
- FOCUS ON DISCOVERY → ENDPOINTS → NETWORK → SCRAPE → AUTH → APPLICATION.
 - ALWAYS CORRELATE WITH POD HEALTH AND CHANGES.

4. COMMON ROOT CAUSES



SERVICE DISCOVERY ISSUE

- Labels changed, service deleted, wrong namespace, servicemonitor selector mismatch.



ENDPOINTS NOT AVAILABLE

- Pods not ready, readiness probe failing, service has no endpoints.



NETWORK ACCESS PROBLEM

- NetworkPolicy blocking traffic, DNS failure, wrong port, service unreachable.



WRONG METRICS PATH

- /metrics changed, wrong path in ServiceMonitor, app exposes metrics on different endpoint.



AUTHENTICATION FAILURE

- Token expired, missing bearer token, basic auth issue, TLS certificates not trusted.



SCRAPE ERRORS

- Timeouts, connection refused, 5xx errors, malformed metrics, unsupported content type.



CARDINALITY EXPLOSION

- Too many labels, high churn, unbounded metrics, Prometheus drops or throttles series.

5. QUICK DEBUG COMMANDS

```

kubectl get pods -n production # Are pods running and ready?
kubectl describe pod <pod> -n production # Check readiness probe and events.
kubectl logs <pod> -n production # Application errors.
curl -v http://<pod-ip>:<port>/metrics # Can you reach metrics endpoint?
kubectl get networkpolicy -n production # Any network blocking traffic?
kubectl get svc,ep -n production # Service and endpoint status.
kubectl describe servicemonitor -n monitoring # Check ServiceMonitor config.
kubectl -n monitoring logs deploy/prometheus # Prometheus scrape errors.

```



SENIOR ENGINEER TIP

PROMETHEUS DOES NOT PUSH. IT ONLY SCRAPES. IF IT CANNOT SEE OR REACH YOUR SERVICE, METRICS WILL DISAPPEAR.



BEST PRACTICES

- KEEP LABELS CONSISTENT.
- MONITOR SCRAPE HEALTH.
- SET ALERTS FOR TARGETS DOWN.

REMEMBER

NO METRICS ≠ SERVICE DOWN. IT COULD BE DISCOVERY, NETWORK, AUTH, PATH, OR CONFIGURATION.



18. CI/CD PIPELINE EXPOSES A SECRET IN BUILD LOGS

SECURITY REMINDER

Hiding the log is NOT the same as containing the compromise.



SCENARIO

An engineer discovers that a production credential appeared in CI logs.

INTERVIEW FOCUS

- ☒ Immediate credential rotation
- ☒ Secret masking
- ☒ Log access
- ☒ CI secret stores
- ☒ Repository history
- ☒ Audit trail
- ☒ Blast-radius assessment
- ☒ IAM permissions
- ☒ Incident response

1. IMMEDIATE ACTIONS (MINUTES MATTER)



1. REVOKE / ROTATE THE SECRET IMMEDIATELY
Assume it is compromised. Do not delay.



2. DISABLE OR RESTRICT ACCESS
If the secret is an API key, token, or user, disable it until investigation is complete.



3. IDENTIFY WHERE IT WAS EXPOSED
Find the pipeline run, job, step, and exact log lines where the secret appeared.



4. COMMUNICATE
Notify the security team, platform team, and affected service owners.



5. ASSESS IMPACT (BLAST RADIUS)
What systems, accounts, environments, and data could this secret access?

2. INVESTIGATION CHECKLIST

- LOG ACCESS
Who has access to the CI logs?
Check retention and log visibility settings.
- REPOSITORY HISTORY
Search commit history for the secret (git log, git blame, branches, forks).
- SECRET STORES & USAGE
Identify all places this secret was used: CI, CD, apps, scripts, containers, IaC, docs.
- AUDIT TRAIL
Review CI/CD platform audit logs and cloud provider activity logs.
- IAM PERMISSIONS
Check what permissions the secret had. Follow least privilege principles.
- DATA EXPOSURE
Determine if the secret could read or modify data or infrastructure.

POSSIBLE IMPACT

- Cloud Account Takeover
- Data Breach
- Service / System Manipulation
- Source Code Access
- Financial Impact

3. PREVENTION: SECRET MASKING

EXAMPLE: GITHUB ACTIONS

```
# Add secret to mask
- name: Mask secret
  run: echo "::add-mask::${{ secrets.MY_SECRET }}"

# Never echo secrets
# Avoid set -x in scripts
```



BEST PRACTICES

- Never print secrets (env, echo, cat, set -x).
- Use native secret masking features.
- Scan logs before exposing them to broader teams.
- Use secrets only at runtime, not build time.
- Avoid passing secrets as command-line arguments.

BLAST-RADIUS QUESTIONS

- What can this secret access?
- Can it modify or delete resources?
- Can it read sensitive data?
- Is the credential long-lived?
- Is it used in multiple environments?
- Could an attacker pivot from this access?



4. INCIDENT RESPONSE FLOW



DETECT
Secret found in CI log



CONTAIN
Rotate / revoke the secret



INVESTIGATE
Find scope and root cause



ERADICATE
Remove exposure, fix pipeline



RECOVER
Validate systems, restore trust



POST-INCIDENT
Document, review, and improve

5. USEFUL COMMANDS

<code>git log -p -S "secret-value-here"</code>	Search commit history for a secret pattern.
<code>git grep -n "secret-value-here"</code>	Search all files in the current repo.
<code>gh secret list</code>	List repository secrets (GitHub CLI).
<code>aws secretsmanager list-secrets</code>	List AWS Secrets Manager secrets.
<code>aws iam list-access-keys --user-name <user></code>	Find IAM access keys.
<code>aws cloudtrail lookup-events --lookup-attributes</code>	Review activity related to the secret.



SECURITY REMINDER: Hiding the log is NOT the same as containing the compromise.
ALWAYS ROTATE. ALWAYS INVESTIGATE. ALWAYS IMPROVE.



19. AUTO SCALING ADDS INSTANCES, BUT PERFORMANCE DOES NOT IMPROVE

SCENARIO

AWS Auto Scaling launches additional instances during traffic spikes, yet latency remains high.

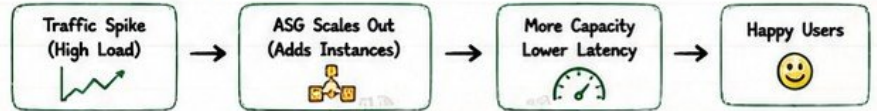
INTERVIEW FOCUS

- ✓ Wrong scaling metric
- ✓ Database bottlenecks
- ✓ Load balancer distribution
- ✓ Application state
- ✓ Session affinity
- ✓ Downstream dependencies
- ✓ Instance warm-up
- ✓ Scaling lag
- ✓ Horizontal scalability limitations

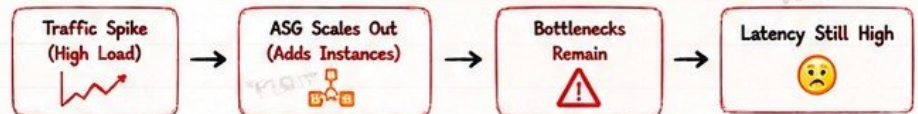


WHAT SHOULD HAPPEN VS. WHAT REALLY HAPPENS

WHAT WE EXPECT



WHAT ACTUALLY HAPPENS



1. COMMON REASONS PERFORMANCE DOES NOT IMPROVE



1. WRONG SCALING METRIC

CPU may be low, but requests, latency, or queue depth are high. Scale on the metric that reflects user experience.



2. DATABASE BOTTLENECKS

DB CPU, connections, locks, or slow queries are the real limit. Adding app instances only increases DB load.



3. LOAD BALANCER DISTRIBUTION

Uneven distribution, unhealthy targets, or cross-AZ latency can cause hot spots.



4. APPLICATION STATE

Stateful apps don't scale linearly. Sessions, caches, or disk content may limit scalability.



5. SESSION AFFINITY

Sticky sessions send users to the same instance. New instances get little or no traffic.



6. DOWNSTREAM DEPENDENCIES

APIs, queues, caches, or third-party services may be slow or rate-limited.



7. INSTANCE WARM-UP

New instances take time to start, download, warm caches, and register in target group.



8. SCALING LAG

CloudWatch metrics + evaluation periods cause delays. Traffic spikes ends before new capacity is ready.



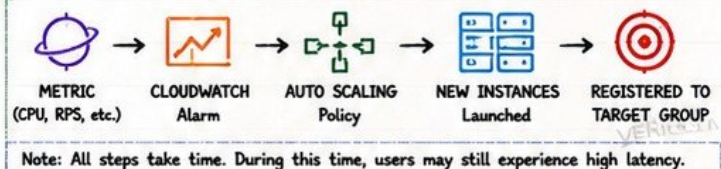
9. HORIZONTAL SCALABILITY LIMITS

Some workloads cannot scale horizontally due to design, locks, or shared resources.

2. DIAGNOSTIC CHECKLIST

- Is the correct metric driving scaling?
- Are new instances passing health checks?
- Is traffic distributed evenly across instances?
- Is the database CPU / connections / IOPS high?
- Are there slow queries or locks in the database?
- Are there errors or timeouts in downstream services?
- Is session affinity enabled on the ALB / NLB?
- Are new instances warming up before receiving traffic?
- Are metrics and scaling policies responsive enough?
- Is the application truly stateless and horizontally scalable?

3. HOW AUTO SCALING WORKS (SIMPLIFIED)



4. INVESTIGATION COMMANDS / TOOLS

AREA	COMMAND / TOOL	WHAT TO CHECK
Auto Scaling	<code>aws autoscaling describe-scaling-activities</code>	Scaling events and timing
Instance Health	<code>aws elbv2 describe-target-health</code>	Targets and health status
Load Balancer	CloudWatch: RequestCount, TargetResponseTime	Latency and distribution
Database	CloudWatch / Performance Insights	CPU, connections, slow queries
Application	APM / Tracing / Logs	Latency, errors, dependencies
Networking	VPC Flow Logs / NACL / SG	Network delays or drops

5. BEST PRACTICES

- Use the right scaling metric (RPS, latency, queue depth).
- Keep applications stateless where possible.
- Use connection pooling and DB read replicas.
- Enable cross-AZ load balancing.
- Warm up instances before sending traffic.
- Remove sticky sessions unless absolutely required.
- Monitor downstream dependencies.
- Test scaling under real-world load.



PRODUCTION NOTE

Auto Scaling adds capacity, but it can only improve performance if the true bottleneck is in the tier being scaled.



SENIOR ENGINEER TIP

Always identify the real bottleneck before scaling. More instances cannot fix a slow database, a bad query, or a broken dependency.



SCALING IS NOT A SOLUTION. SCALING THE WRONG THING IS AN EXPENSIVE PROBLEM.

20. KUBERNETES POD IS STUCK IN PENDING

SCENARIO

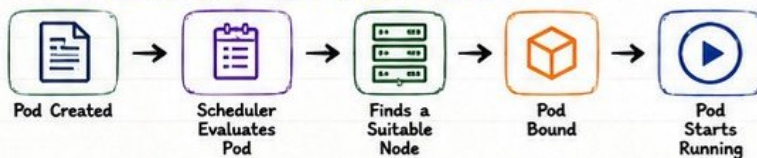
A newly created pod remains Pending indefinitely.

INTERVIEW FOCUS

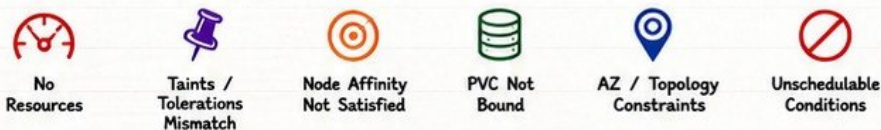
- ✓ Scheduler events
- ✓ Resource requests
- ✓ Node capacity
- ✓ Taints and tolerations
- ✓ Node affinity
- ✓ PVC binding
- ✓ Availability Zones
- ✓ Pod topology constraints
- ✓ Unschedulable conditions



POD SCHEDULING FLOW (HIGH LEVEL)



WHY POD STAYS PENDING



1. CHECK THE POD STATUS & EVENTS

```
kubectl get pod <pod-name> -n <namespace>
```

NAME	READY	STATUS	RESTARTS	AGE
myapp-7d5f8c9b6c-abc12	0/1	Pending	0	2m

```
kubectl describe pod <pod-name> -n <namespace>
```

Look for Events at the bottom.

COMMON EVENTS

- 0/5 nodes are available: insufficient cpu.
- 0/5 nodes are available: insufficient memory.
- 0/5 nodes are available: node(s) had taint {key: value}.
- pod has unbound immediate PersistentVolumeClaims.
- node(s) didn't match pod affinity/anti-affinity rules.
- node(s) didn't match pod topology spread constraints.

2. CHECK NODE CAPACITY & RESOURCES

```
kubectl get nodes
```

```
kubectl top nodes
```

Compare Pod requests with available resources.

RESOURCE	CHECK WHAT	COMMAND / TOOL
CPU	Are nodes low on allocatable CPU?	kubectl top nodes kubectl describe node
MEMORY	Are nodes low on allocatable memory?	kubectl top nodes kubectl describe node
STORAGE	Is disk pressure blocking scheduling?	kubectl describe node Check conditions

9. USEFUL COMMANDS QUICK REFERENCE

kubectl get pods -A	List all pods in all namespaces
kubectl get events -A --sort-by=.lastTimestamp	View recent events cluster-wide
kubectl describe pod <pod>	Detailed pod info and events
kubectl get nodes -o wide	See node zones and status
kubectl get pvc -A	Check PVC status
kubectl top nodes	Check resource usage
kubectl describe node <node>	Check node conditions and taints

3. CHECK TAINTS & TOLERATIONS

```
kubectl describe node <node-name> | grep -A2 Taints
```

If node has taints, pod must tolerate them.

NODE TAINT	POD NEEDS TOLERATION
key=value:NoSchedule	toleration with equal key/value and effect NoSchedule
key=value:NoExecute	toleration with equal key/value and effect NoExecute
key=value:PreferNoSchedule	toleration with equal key/value and effect PreferNoSchedule

4. CHECK NODE AFFINITY / ANTI-AFFINITY

```
kubectl get pod <pod-name> -n <namespace> -o yaml
```

Review:

- nodeSelector
- nodeAffinity
- requiredDuringSchedulingIgnoredDuringExecution
- preferredDuringSchedulingIgnoredDuringExecution
- podAntiAffinity / podAffinity



If no nodes match the rules, the pod stays Pending.

5. CHECK PVC BINDING

```
kubectl get pvc -n <namespace>
```

NAME	STATUS	VOLUME	CAPACITY	STORAGECLASS	AGE
data-pvc	Pending	-	10Gi	gp3	2m

If PVC is Pending → Pod cannot be scheduled.

Check StorageClass, availability in correct AZ, and dynamic provisioning.

6. CHECK AVAILABILITY ZONES

If nodes and PVCs are in different AZs, pod cannot be scheduled.

```
kubectl get nodes -o wide
```

NAME	ZONE	STATUS
ip-10-0-1-10	us-east-1a	Ready
ip-10-0-2-15	us-east-1b	Ready
ip-10-0-3-20	us-east-1c	Ready



Ensure PVC, node, and topology rules align in the same AZ.

7. CHECK POD TOPOLOGY CONSTRAINTS

```
kubectl get pod <pod-name> -o yaml
```

Look for:

- topologySpreadConstraints
- maxSkew
- topologyKey
- whenUnsatisfiable



These rules can block scheduling if constraints cannot be satisfied.

8. UNSCHEDULABLE CONDITIONS TO LOOK FOR

- Insufficient CPU / Memory / Pods
- Node(s) didn't match node selector
- Node(s) didn't match affinity rules
- Pod has unbound PVC
- Taints not tolerated
- Topology constraints unsatisfied
- All nodes cordoned or unschedulable
- Cluster autoscaler not adding nodes



Any of the above keeps the pod in Pending.



PRODUCTION NOTE

- Pending pods = unsatisfied requirement.
- Events tell you WHY.
- Fix the cause, not the symptom.
- Avoid disabling rules unless necessary.
- Automation (Cluster Autoscaler) helps, but correct configuration is key.



SENIOR ENGINEER TIP

Always read the Events. They point directly to the root cause. Think like the scheduler: Would this pod fit anywhere in the cluster right now? If not, WHY not?



PENDING PODS ARE NOT RANDOM. THEY ARE TELLING YOU WHAT THEY NEED. READ EVENTS. CHECK REQUIREMENTS. FIX THE ROOT CAUSE.



PAGE 21: APPLICATION CANNOT RESOLVE INTERNAL DNS NAMES

SCENARIO:

Containers can communicate using IP addresses but fail when using service names.

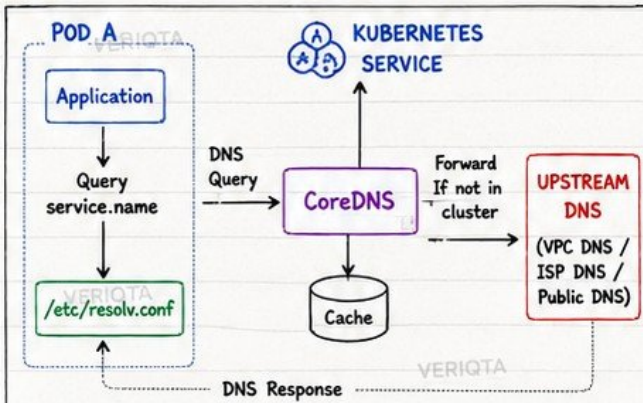
INTERVIEW FOCUS

- ✓ CoreDNS
- ✓ resolv.conf
- ✓ Kubernetes DNS
- ✓ Search domains
- ✓ NetworkPolicy
- ✓ DNS upstreams
- ✓ CoreDNS logs
- ✓ nslookup and dig
- ✓ DNS packet troubleshooting

REMEMBER

DNS problems look like network problems, but the root cause is usually DNS.

1. HOW KUBERNETES DNS WORKS



FLOW:

1. Pod sends DNS query to CoreDNS (Cluster IP)
2. CoreDNS checks cache
3. If found → returns response
4. If not found → forwards to upstream DNS
5. Response is returned to the pod

2. CHECK /etc/resolv.conf INSIDE THE POD

```
# kubectl exec -it <pod-name> -- cat /etc/resolv.conf

nameserver 10.96.0.10      ← Cluster DNS (CoreDNS Service IP)
search default.svc.cluster.local svc.cluster.local cluster.local ← Search domains
options ndots:5           ← Try search domains first
```

EXPLANATION:

- nameserver → Where DNS queries go
- search → Domains appended to short names
- ndots:5 → If hostname has <5 dots, search domains are tried first

3. QUICK DIAGNOSTIC COMMANDS INSIDE THE POD

COMMAND	PURPOSE
nslookup service-name	Basic DNS resolution test
dig service-name	Detailed DNS query
dig @10.96.0.10 service-name	Query CoreDNS directly
dig +trace service-name	See full DNS resolution path
cat /etc/resolv.conf	Check DNS configuration
ping service-name	Tests DNS + network together

4. COMMON ROOT CAUSES

- ✗ CoreDNS pods are down or crashlooping
- ✗ CoreDNS ConfigMap misconfiguration
- ✗ NetworkPolicy blocking DNS (UDP/TCP 53)
- ✗ Upstream DNS is unreachable
- ✗ Node / VPC DNS issues
- ✗ Incorrect search domains
- ✗ Stale CoreDNS cache
- ✗ Service not created or wrong namespace

If IP works but name doesn't, think DNS first!

5. CHECK COREDNS HEALTH

```
# kubectl get pods -n kube-system -l k8s-app=kube-dns
# kubectl logs -n kube-system -l k8s-app=kube-dns >
# kubectl describe pod -n kube-system <coredns-pod>
```

GOOD SIGNS

- No CrashLoopBackOff
- Logs show "plugin ready"
- No SERVFAIL errors

6. NETWORK POLICY CHECK

DNS uses UDP/TCP port 53
Ensure pods can reach CoreDNS.

```
# kubectl get networkpolicy -A
# kubectl describe networkpolicy <policy-name>
# Allow egress to 10.96.0.10 on port 53
```

7. DNS PACKET TROUBLESHOOTING

```
# tcpdump -n -i any port 53
```

Look for:

- Outgoing DNS query
- Incoming DNS response

No response = DNS is blocked or upstream issue.

PRODUCTION NOTE

DNS failures can cause total outages even when all services and pods look healthy. Always check DNS early in the troubleshooting process.



SENIOR ENGINEER TIP

Always test from the same namespace as the failing pod. Different namespaces can have different NetworkPolicies or DNS behavior.



REAL WORLD LESSON

Most "application bugs" in Kubernetes turn out to be DNS misconfigurations or blocked DNS traffic.



GOLDEN RULE

If you can't resolve the name, the application will fail. Fix DNS, fix the app.

PAGE 22: PRODUCTION SERVER HAS PLENTY OF MEMORY BUT APPLICATIONS CANNOT START NEW PROCESSES

A JUST BECAUSE MEMORY AND DISK ARE FINE DOES NOT MEAN THE SYSTEM CAN CREATE NEW PROCESSES.

SCENARIO:

Memory and disk capacity appear normal, but new processes fail to start.

INTERVIEW FOCUS

- PID exhaustion
- ulimit
- File descriptors
- Process limits
- Zombie processes
- Kernel limits
- systemd limits
- Resource investigation

1. WHAT DOES THE ERROR LOOK LIKE?

COMMON ERRORS:

- fork: retry: Resource temporarily unavailable
- cannot fork
- Resource temporarily unavailable
- pthread_create failed
- shell unable to create child process



SYSTEM IS OUT OF PROCESS SLOTS!

2. CHECK PID USAGE (MOST COMMON ROOT CAUSE)

Check current PID usage

cat /proc/sys/kernel/pid_max ← Max PIDs allowed

ps -eLf | wc -1 ← Current number of PIDs

ps -eo stat | awk '{print \$1}' | sort | uniq -c | sort -nr | head

← See many zombie (Z) processes?

EXPLANATION:

If current PIDs are close to pid_max, the system cannot create new processes even if memory and disk are fine.

3. CHECK ULIMITS (PER-USER LIMITS)

COMMAND	PURPOSE
ulimit -a	Show all current limits
ulimit -u	Max user processes (nproc)
ulimit -n	Max open files (file descriptors)
ulimit -s	Stack size
ulimit -c	Core file size
ulimit -p	Max locked memory

4. CHECK FILE DESCRIPTOR USAGE

System wide open file descriptors

lsof | wc -1 ← Total open files

Top processes by open files

lsof -nP | awk '{print \$2}' | sort | uniq -c | sort -nr | head

Per process open files

lsof -p <PID> | wc -1 ← Replace <PID>



TOO MANY OPEN FILE DESCRIPTORS CAN BLOCK NEW PROCESSES.

5. CHECK ZOMBIE PROCESSES

List zombie processes

ps -eo pid,ppid,stat,cmd | grep 'Z' ← Z = zombie

Count zombies

ps -eo stat | grep -c 'Z'



ZOMBIES CONSUME PID SLOTS!

6. CHECK KERNEL PROCESS LIMITS

FILE	PURPOSE
/proc/sys/kernel/pid_max	Maximum PIDs the system can create
/proc/sys/kernel/threads-max	Maximum number of threads
/proc/sys/kernel/nr_threads	Current number of threads
/proc/sys/kernel/ns_last_pid	Last used PID

7. CHECK SYSTEMD LIMITS (FOR SERVICES)

Show limits for a service

systemctl show <service> -p LimitNPROC -p LimitNOFILE -p TasksMax

Example output

LimitNPROC=16384

← Max user processes

LimitNOFILE=65535

← Max open files

TasksMax=512

← Max tasks (processes + threads)

DefaultTasksMax=512

← System default limit



SYSTEMD CAN SILENTLY LIMIT YOUR PROCESSES!

8. QUICK RESOURCE INVESTIGATION CHECKLIST

- ✓ CHECK PID USAGE (IS IT NEAR pid_max?)
- ✓ CHECK ULIMIT -u (MAX USER PROCESSES)
- ✓ CHECK FILE DESCRIPTOR USAGE (ULIMIT -n, lsof)
- ✓ CHECK FOR ZOMBIE PROCESSES
- ✓ CHECK KERNEL THREAD LIMITS
- ✓ CHECK SYSTEMD SERVICE LIMITS
- ✓ REVIEW APPLICATION SPAWN PATTERNS AND LOGS



REAL WORLD LESSON

PID exhaustion is common in servers with:

- Short-lived processes
- Fork bombs
- Poorly written applications
- Unreaped child processes
- Aggressive monitoring scripts



SENIOR ENGINEER TIP

Always monitor and alert on:

- PID usage percentage
- Open file descriptors
- Thread count
- Zombie process count
- Systemd TasksMax



PRODUCTION ADVICE

Do not blindly increase kernel limits. First, identify why the system is creating too many processes or keeping them around. Fix the root cause. Then tune the limits safely.

PAGE 23: ARGO CD SHOWS OUTFOSYNC IMMEDIATELY AFTER EVERY SYNC

SCENARIO:

Argo CD successfully synchronizes an application, but it quickly returns to OutOfSync.

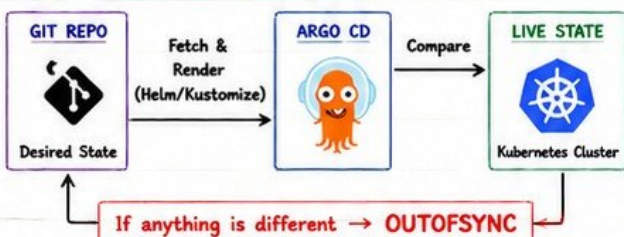
INTERVIEW FOCUS

- Desired versus live state
- Mutating controllers
- Defaulted fields
- Helm rendering
- IgnoreDifferences
- Operators
- Admission webhooks
- Manual cluster changes
- GitOps ownership

REMEMBER

Argo CD is not failing. It is detecting drift between Desired (Git) and Live (cluster). Find out why the cluster keeps changing!

1. HOW ARGO CD DETECTS DRIFT



2. COMMON REASONS FOR PERMANENT OUTFOSYNC

REASON	DESCRIPTION
Mutating Controllers	Controllers modify objects after Argo CD syncs them.
Defaulted Fields	Kubernetes adds default values not in Git manifest.
Helm Rendering	Helm template functions create dynamic values.
IgnoreDifferences Missing	Argo CD compares fields that should be ignored.
Operators	Operators reconcile and change resource fields.
Admission Webhooks	Webhooks inject or mutate resources.
Manual Cluster Changes	Someone changed the resource outside Git.
GitOps Ownership	Argo CD should own the resource, not humans.

3. TROUBLESHOOTING FLOW

1. Check Argo CD application status
`argocd app get <app-name>`
2. Inspect the diff
`argocd app diff <app-name>`
3. Identify which fields are different
4. Find the root cause (see section 2)
5. Either fix the cause or ignore expected differences

CHECK FIRST:

- Is the diff expected?
 - Is a controller mutating it?
 - Is Kubernetes adding defaults?
 - Is Helm rendering dynamic values?
 - Is someone changing the resource?
 - Does Argo CD actually own this resource?
- Don't ignore drift blindly. Understand it first!

4. EXAMPLE: MUTATING CONTROLLER ADDING LABELS

DESIRED (GIT)	LIVE (CLUSTER)
<pre>apiVersion: apps/v1 kind: Deployment metadata: name: my-app labels: app: my-app spec: replicas: 3 selector: matchLabels: app: my-app template: metadata: labels: app: my-app</pre>	<pre>apiVersion: apps/v1 kind: Deployment metadata: name: my-app labels: app: my-app pod-template-hash: 7d8f9c6d5 argocd.argoproj.io/instance: my-app spec: replicas: 3 selector: matchLabels: app: my-app template: metadata: labels: app: my-app pod-template-hash: 7d8f9c6d5</pre>

OUTFOSYNC because of extra labels added by the system

5. HOW TO IGNORE EXPECTED DIFFERENCES

Example: Ignore auto-generated labels and annotations spec:

```
ignoreDifferences:
- group: "apps"
  kind: Deployment
  name: my-app
  jsonPointers:
  - /metadata/labels/pod-template-hash
  - /metadata/labels/argocd.argoproj.io~1instance
  - /metadata/annotations
```

Use jsonPointers for precise fields. Avoid ignoring too much!

6. HELM RENDERING CAN CAUSE DRIFT

HELM TEMPLATE (GIT)	RENDERED VALUE CHANGES EVERY SYNC
<pre>metadata: annotations: checksum/config: "{{ .Values.checksum }}"</pre>	<pre>checksum/config: "a1b2c3d4" checksum/config: "e5f6g7h8" checksum/config: "i9j0k1l2"</pre>

Each render produces a new value -> DRIFT!

SOLUTION	IGNORE EXAMPLE
• Use stable values • Ignore differences for dynamic fields • Or move dynamic values to a controller	<pre>jsonPointers: - /metadata/annotations/checksum~1config</pre>

7. USEFUL ARGO CD COMMANDS

COMMAND	PURPOSE
<code>argocd app get <app-name></code>	Show app status and sync state
<code>argocd app diff <app-name></code>	Show differences between desired and live
<code>argocd app manifests <app-name></code>	Show rendered manifests (desired)
<code>argocd app history <app-name></code>	Show sync history
<code>argocd app resources <app-name></code>	List resources managed by the app
<code>argocd app sync <app-name></code>	Sync the application
<code>argocd app refresh <app-name></code>	Refresh live state
<code>kubectl get events -A --sort-by=.LastTimestamp</code>	Check for controller/webhook events
<code>kubectl get mutatingwebhookconfigurations</code>	List mutating webhooks
<code>kubectl get pods -A -o wide</code>	Check operator/controller pods

REAL WORLD LESSON

Most OutOfSync issues are not Argo CD bugs. They are normal Kubernetes behavior. Your job is to decide what should be ignored and what should be fixed.

SENIOR ENGINEER TIP

Never ignore differences you don't understand. Silent drift can hide real problems and security risks.

PRODUCTION ADVICE

Keep Git as the single source of truth. If Argo CD doesn't own it, it will never stay in sync.

BEST PRACTICE

- Define ownership clearly
- Use IgnoreDifferences wisely
- Monitor drift continuously
- Understand the root cause

PAGE 24: KUBERNETES ROLLING DEPLOYMENT CAUSES A PARTIAL OUTAGE

SCENARIO:

A deployment uses RollingUpdate, yet users receive errors while new pods replace old ones.

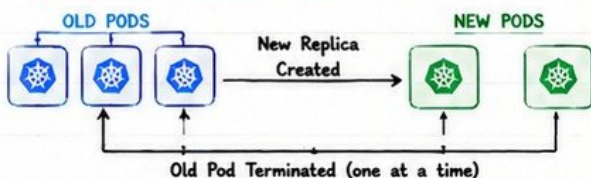
INTERVIEW FOCUS

- ✓ Readiness probes
- ✓ maxUnavailable
- ✓ maxSurge
- ✓ Graceful shutdown
- ✓ terminationGracePeriodSeconds
- ✓ preStop
- ✓ Load balancer deregistration
- ✓ Connection draining
- ✓ Application startup time

REMEMBER

Kubernetes does not know about your application connections. YOU must make sure old pods exit safely before traffic stops.

1. HOW ROLLINGUPDATE WORKS (DEFAULT)



If the new pod is not ready in time, users will hit errors during the transition.

2. KEY ROLLINGUPDATE STRATEGY SETTINGS

SETTING	DEFAULT	PURPOSE
maxUnavailable	25%	Max number of pods that can be unavailable during update.
maxSurge	25%	Max number of extra pods created above desired replicas.
progressDeadlineSeconds	600	Time before the deployment is marked as failed.



Lower maxUnavailable = more availability.
Lower maxSurge = less extra capacity needed.
Find the right balance for your app.

3. SAMPLE DEPLOYMENT STRATEGY

```
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1 # At most 1 pod unavailable
      maxSurge: 1 # At most 1 extra pod
      minReadySeconds: 10 # Pod must be ready for 10s
      progressDeadlineSeconds: 600
      replicas: 5
```

4. POD LIFECYCLE DURING ROLLING UPDATE



If readiness probe is slow or startup time is long, old pod may be removed before new pod is ready → OUTAGE

5. GRACEFUL SHUTDOWN CONFIGURATION

```
spec:
  template:
    spec:
      containers:
        - name: app
          image: my-app:1.0
          lifecycle:
            preStop:
              exec:
                command: ["/bin/sh", "-c", "sleep 15"] # Give time to drain
            terminationGracePeriodSeconds: 30 # Total time before kill
```

WHY preStop?

It gives your app a chance to stop accepting new requests and finish existing ones before the pod is killed.

6. LOAD BALANCER & CONNECTION DRAINING

ENSURE YOUR LOAD BALANCER

- Removes pod from rotation on termination
- Waits for connection drain
- Health check interval < preStop sleep time
- Deregistration delay is configured



If the LB keeps sending traffic to a terminating pod, users will see errors or timeouts.

7. COMMON CAUSES OF PARTIAL OUTAGES

CAUSE	WHAT HAPPENS	HOW TO FIX
Slow application startup	New pods not ready in time	Optimize startup, increase probes, use minReadySeconds
Readiness probe too slow	Pod added late → traffic gap	Make probes fast and reliable
maxUnavailable too high	Too many pods down	Set to 0 or 1 for critical apps
No preStop / short grace	Connections cut off abruptly	Add preStop + increase terminationGracePeriodSeconds
LB deregistration delay low	LB still sends traffic	Increase deregistration delay / drain time
No connection draining in app	Active connections reset	Implement graceful shutdown in app
DB / dependencies slow	App takes longer to become ready	Check dependencies, use readiness checks

8. CHECKLIST FOR ZERO-DOWNTIME DEPLOYMENTS

- ✓ Readiness probe is accurate and fast
- ✓ Startup time < pod termination time
- ✓ minReadySeconds is set
- ✓ maxUnavailable is small (0 or 1)
- ✓ maxSurge is appropriate
- ✓ preStop hook is implemented
- ✓ terminationGracePeriodSeconds is sufficient
- ✓ Load balancer deregistration delay is set
- ✓ Monitor rollout with kubectl rollout status
- ✓ Test deployments in staging first



★ REAL WORLD LESSON

RollingUpdate is safe only when your application and infrastructure are ready for it. Kubernetes provides the mechanism, you provide the timing.



SENIOR ENGINEER TIP

Always think in terms of connections, not just pods. Protect users from the gap between old pod termination and new pod readiness.



PRODUCTION ADVICE

Observe real deployments. Tune probes, grace periods, and LB settings based on real traffic and behavior.



BEST PRACTICE

Make your app graceful by design. Fast startup, quick readiness, clean shutdown.

PAGE 25: AWS RDS DATABASE SUDDENLY BECOMES SLOW

SCENARIO:

The database is available, but application latency increases significantly.

VERIQTA

INTERVIEW FOCUS

- | | |
|----------------|--|
| ✓ CPU | ✓ Performance Insights |
| ✓ Memory | ✓ Storage throughput |
| ✓ IOPS | ✓ Read replicas |
| ✓ Connections | ✓ Application query behavior |
| ✓ Slow queries | ✓ Database versus infrastructure diagnosis |
| ✓ Locks | |

REMEMBER

A slow database is not always an infrastructure issue. Always prove where the bottleneck is. Measure. Don't guess. Then act.

1. FIRST: CONFIRM THE PROBLEM AND IMPACT

	Verify RDS status is available (AWS Console / describe-db-instances)
	Confirm application latency increase (APM / logs / metrics)
	Identify time window of degradation
	Check if issue is isolated to one DB or multiple
	Check recent changes (deployments, schema changes, parameter changes)

2. CHECK KEY RDS METRICS (CLOUDWATCH)

VERIQTA METRIC	WHAT IT MEANS	POSSIBLE ISSUE WHEN HIGH
CPUUtilization	CPU usage on the DB instance	High CPU from queries or load
DatabaseConnections	Active connections	Too many connections
FreeableMemory	Available memory	Low memory pressure
ReadIOPS / WriteIOPS	Disk I/O operations	I/O bottleneck
ReadLatency / WriteLatency	I/O latency	Storage or I/O issues
FreeStorageSpace	Remaining storage	Low space causes slowness
NetworkThroughput	Network in/out	Network saturation
ReplicaLag	Read replica lag	Replicas falling behind



Correlate metric spikes with the time window of the slowdown.

3. CHECK CONNECTIONS

MYSQL / MARIADB
SHOW PROCESSLIST;
SHOW STATUS LIKE 'Threads_connected';
SHOW STATUS LIKE 'Max_used_connections';

POSTGRESQL
SELECT count(*) FROM pg_stat_activity;
SELECT * FROM pg_stat_activity
WHERE state != 'idle';



Too many connections can exhaust resources and block new requests.

4. FIND SLOW QUERIES

USING PERFORMANCE INSIGHTS

- Go to RDS Console → Performance Insights
- View top SQL by load (DB Load)
- Check Wait Events
- Identify SQL causing high load

ENABLE SLOW QUERY LOG (IF NOT ALREADY ENABLED)

MYSQL / MARIADB
SET GLOBAL slow_query_log = 'ON';
SET GLOBAL long_query_time = 1;

POSTGRESQL
log_min_duration_statement = 1000 # 1s



Slow queries are the #1 cause of database slowness.

5. CHECK LOCKS AND BLOCKING

MYSQL / MARIADB
SHOW ENGINE INNODB STATUS\G
SELECT * FROM information_schema.innodb_locks;
SELECT * FROM information_schema.innodb_trx;

POSTGRESQL
SELECT * FROM pg_locks;
SELECT * FROM pg_stat_activity
WHERE wait_event_type IS NOT NULL;



Locks and long transactions can block other queries and cause high latency.

6. CHECK STORAGE AND I/O

WHAT TO CHECK	WHY IT MATTERS
IOPS	High I/O load can cause latency
Read/Write Latency	High latency slows query response
StorageThroughput	Low throughput = bottleneck
Burst Balance (gp2/gp3)	Low balance limits performance
EBS Volume Type	General Purpose vs Provisioned IOPS

CHECK EBS METRICS (CLOUDWATCH)

- VolumeQueueLength
- VolumeWriteLatency
- VolumeReadLatency
- VolumeThroughputPercentage



7. DATABASE VS INFRASTRUCTURE DIAGNOSIS

LIKELY DATABASE (QUERY) ISSUE

- High CPU with low I/O
- Many slow queries
- Locks and long transactions
- High connections
- Performance Insights shows SQL as root cause
- Query plan inefficiency

VS

LIKELY INFRASTRUCTURE ISSUE

- High IOPS with high latency
- High read/write latency
- Low storage throughput
- Low burst balance
- Network throughput issues
- CPU low but latency high



Always prove the root cause before scaling or changing infrastructure.

8. CHECK READ REPLICAS

- Check ReplicaLag
- Ensure replicas are healthy
- Check replica CPU and I/O
- Offload read traffic from primary
- Promote a replica only if necessary

9. APPLICATION QUERY BEHAVIOR



- N+1 queries
- Missing indexes
- Full table scans
- Inefficient joins
- Incorrect connection pooling
- ORMS generating heavy queries

QUICK INVESTIGATION CHECKLIST

- | | |
|----------------------------------|--------------------------|
| ✓ Confirm impact and time window | ✓ Check storage & I/O |
| ✓ Check CloudWatch metrics | ✓ Check read replicas |
| ✓ Check connections | ✓ Analyze query behavior |
| ✓ Find slow queries | ✓ Determine root cause |
| ✓ Check locks | ✓ Fix and verify |



REAL WORLD LESSON

Most slow RDS issues are caused by queries, not hardware. Measure, isolate, and fix the real bottleneck.



SENIOR ENGINEER TIP

Use Performance Insights first. It shows the truth faster than guessing.



PRODUCTION ADVICE

Do not restart or scale before identifying the actual root cause.



BEST PRACTICE

Continuously monitor, optimize queries, and set alerts on key RDS metrics.

26. DOCKER IMAGE IS 2 GB AND DEPLOYMENTS ARE SLOW

SCENARIO:

CONTAINER DEPLOYMENTS TAKE SEVERAL MINUTES BECAUSE IMAGES HAVE BECOME EXTREMELY LARGE.

INTERVIEW FOCUS

- ✓ MULTI-STAGE BUILDS
- ✓ BASE IMAGES
- ✓ LAYER ORDERING
- ✓ BUILD CONTEXT
- ✓ .DOCKERIGNORE
- ✓ PACKAGE CACHES
- ✓ DEPENDENCY CLEANUP
- ✓ SECURITY IMPLICATIONS
- ✓ IMAGE PULL LATENCY
- ✓ REPRODUCIBLE BUILDS



WHY LARGE IMAGES HURT

- LONGER PULL TIMES
- HIGHER NETWORK USAGE
- SLOWER DEPLOYMENTS
- MORE STORAGE CONSUMPTION
- BIGGER ATTACK SURFACE
- HARDER TO SCAN
- MORE EXPENSIVE IN REGISTRIES & CI/CD

LARGE IMAGE
2.0 GB



OPTIMIZED IMAGE
180 MB



FASTER PULLS • FASTER DEPLOYS • LOWER COSTS
SMALLER SURFACE • BETTER SECURITY

USE MULTI-STAGE BUILDS

BAD: SINGLE STAGE (LARGE)

```
FROM node:18
WORKDIR /app
COPY . .
RUN npm install
RUN npm run build
EXPOSE 3000
CMD ["node", "server.js"]
```

→ ~ 2.0 GB

GOOD: MULTI-STAGE (SMALL)

```
FROM node:18 AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
RUN npm run build

FROM node:18-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
EXPOSE 3000
CMD ["node", "dist/server.js"]
```

→ ~180 MB

KEY OPTIMIZATION TECHNIQUES



CHOOSE SMALL BASE IMAGES

Prefer alpine, distroless, or slim variants.



ORDER LAYERS WISELY

Put rarely changing files first to leverage Docker cache.



MINIMIZE BUILD CONTEXT

Send only necessary files to Docker.



USE .DOCKERIGNORE

Exclude node_modules, .git, logs, dist, tests, docs, local files, etc.



CLEAN UP IN THE SAME LAYER

Remove package caches and temp files in the same RUN layer.



SECURITY MATTERS

Smaller images have fewer vulnerabilities and are faster to scan.

COMMON MISTAKES

- ✗ COPY . . (copies everything including .git, node_modules)
- ✗ Installing dev dependencies in the final image
- ✗ Not cleaning package caches (apt, yum, apk, pip, npm)
- ✗ Using huge base images without reason
- ✗ Running unnecessary tools in production image
- ✗ Forgetting .dockerignore
- ✗ Invalidating cache due to wrong layer order



PRODUCTION NOTE



A FAST, SMALL IMAGE IS NOT JUST ABOUT SPEED. IT'S ABOUT RELIABILITY, SECURITY, COST, AND OPERATIONAL EXCELLENCE. TREAT YOUR DOCKERFILE LIKE PRODUCTION CODE.



USEFUL COMMANDS

```
# Image size
docker images

# Inspect layers
docker history <image>

# Analyze image
docker scout cves <image>

# Check build context
docker build --no-cache -t test .

# Estimate pull time
crane ls <image> | wc -l
```



REAL ENGINEERS OPTIMIZE IMAGES.
GREAT ENGINEERS OPTIMIZE EVERYTHING.



SMALL IMAGES
FAST DEPLOYS
HAPPY USERS



REMEMBER

EVERY MB YOU ELIMINATE SAVES TIME, MONEY, AND TROUBLE.



— VERIQTA —

27. PRODUCTION HTTPS CERTIFICATE EXPIRES

SCENARIO:

USERS SUDDENLY RECEIVE CERTIFICATE ERRORS WHEN ACCESSING THE PRODUCTION APPLICATION.

INTERVIEW FOCUS

- ✓ CERTIFICATE CHAIN
- ✓ EXPIRATION
- ✓ AUTOMATED RENEWAL
- ✓ ACM (AWS CERTIFICATE MANAGER)
- ✓ LET'S ENCRYPT
- ✓ LOAD BALANCER TERMINATION
- ✓ KUBERNETES CERT-MANAGER
- ✓ MONITORING EXPIRATION
- ✓ EMERGENCY CERT REPLACEMENT



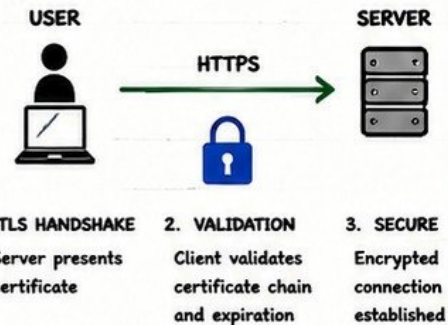
WHY CERTIFICATE EXPIRY IS CRITICAL

- USERS SEE ERRORS (ERR_CERT_DATE_INVALID)
- BROWSER BLOCKS ACCESS
- API CLIENTS FAIL SECURE CONNECTION
- REVENUE AND TRUST IMPACTED
- INCIDENT ESCALATES FAST

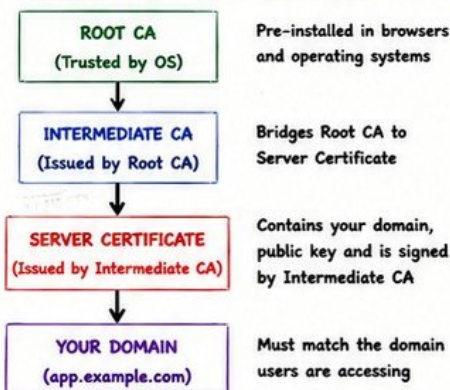
COMMON USER ERRORS

- This site can't provide a secure connection
- Your connection is not private
- NET::ERR_CERT_DATE_INVALID
- CERTIFICATE_VERIFY_FAILED

HOW HTTPS WORKS (SIMPLIFIED)



CERTIFICATE CHAIN EXAMPLE



EXPIRATION CHECK COMMANDS



LINUX / MAC

```
openssl s_client -connect app.example.com:443 \
  -servername app.example.com 2>/dev/null | \
  openssl x509 -noout -dates
```



WINDOWS (POWERSHELL)

```
openssl s_client -connect app.example.com:443 \
  -servername app.example.com | \
  Select-String "notAfter"
```



CHECK CERTIFICATE CHAIN

```
openssl s_client -connect app.example.com:443 \
  -showcerts 2>/dev/null
```

MONITOR CERTIFICATE EXPIRATION

- SET ALERTS TO WARN 30, 15, 7, 3 DAYS BEFORE EXPIRY
- MONITOR ALL ENVIRONMENTS
- INCLUDE LOAD BALANCERS, API GATEWAYS, INGRESSES, CDN AND THIRD-PARTY CERTS
- USE CERTIFICATE TRANSPARENCY LOGS
- EXAMPLES: PROMETHEUS BLACKBOX EXPORTER, GRAFANA, PAGERDUTY, DATADOG

MONITORING EXAMPLE (PROMETHEUS ALERT)

ALERT: CertificateExpiringSoon
 IF probe_ssl_earliest_cert_expiry - time() < 604800
 FOR 10m
 LABELS: severity="warning"
 ANNOTATIONS:
 summary: "Certificate expires in {{ \$value | humanizeDuration }}"



AWS ACM (AUTOMATED)



- MANAGED BY AWS
- AUTO RENEWAL
- INTEGRATES WITH ALB, NLB, CLOUD FRONT, API GW
- PRIVATE KEY MANAGED BY AWS

LET'S ENCRYPT (AUTOMATED)



- FREE SSL/TLS
- AUTO RENEWAL (90 DAYS)
- WIDELY SUPPORTED
- USE CERTBOT ON SERVERS OR CERT-MANAGER IN KUBERNETES

LOAD BALANCER TERMINATION



- TLS TERMINATES AT LB
- CERT MANAGED BY LB
- BACKEND TRAFFIC CAN BE HTTP OR HTTPS
- ENSURE CERT IS ATTACHED TO THE CORRECT LISTENER

KUBERNETES CERT-MANAGER



- AUTOMATES CERTS IN K8S
- SUPPORTS LET'S ENCRYPT, VAULT, AWS ACM
- HANDLES RENEWAL
- WORKS WITH INGRESS, GATEWAY API, SERVICES

OTHER PLATFORMS



- AZURE KEY VAULT + APIM
- GCP MANAGED CERTS
- CLOUDFLARE SSL
- CDN MANAGED CERTS
- THIRD-PARTY CERTS (MONITOR THEM TOO)



EMERGENCY CERTIFICATE REPLACEMENT

1. IDENTIFY WHERE TLS TERMINATES (LB / INGRESS / SERVER / CDN)
2. GENERATE OR REQUEST NEW CERTIFICATE
3. VALIDATE CERTIFICATE AND CHAIN
4. DEPLOY TO THE CORRECT RESOURCE
5. RELOAD / RESTART IF REQUIRED
6. VERIFY FROM MULTIPLE REGIONS AND BROWSERS
7. MONITOR FOR ANY ISSUES



PRODUCTION NOTE

CERTIFICATES ARE EASY TO FORGET BUT IMPOSSIBLE TO IGNORE. AUTOMATE, MONITOR, AND TEST RENEWALS REGULARLY. A EXPIRED CERT IS A PREVENTABLE OUTAGE.

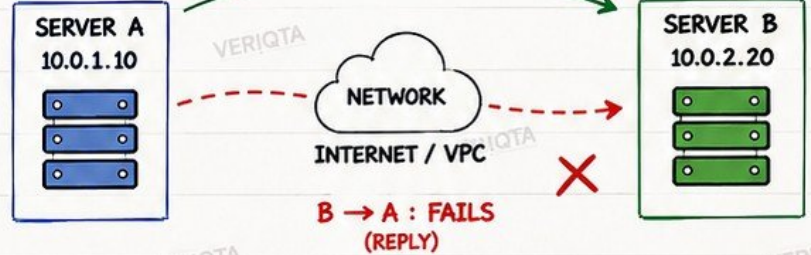


28. NETWORK CONNECTIVITY WORKS ONE WAY BUT NOT THE OTHER

SCENARIO: SERVER A CAN INITIATE COMMUNICATION WITH SERVER B, BUT TRAFFIC INITIATED FROM B TO A FAILS.

INTERVIEW FOCUS

- ✓ STATEFUL VS STATELESS FILTERING
- ✓ SECURITY GROUPS
- ✓ NACLs
- ✓ ROUTE TABLES
- ✓ FIREWALLS
- ✓ ASYMMETRIC ROUTING
- ✓ RETURN PATHS
- ✓ TCPDUMP
- ✓ TRACEROUTE
- ✓ NETWORK TROUBLESHOOTING METHODOLOGY



COMMON CAUSES

- ✗ STATELESS FILTER (NACL/FIREWALL) BLOCKS RETURN TRAFFIC
- ✗ MISSING RETURN ROUTE
- ✗ ASYMMETRIC ROUTING
- ✗ INCORRECT SECURITY GROUP RULES
- ✗ FIREWALL DEFAULT DENY ON INBOUND
- ✗ PATH MTU / FRAGMENTATION ISSUES
- ✗ RP FILTER / ANTI-SPOOFING DROPS PACKETS

STATEFUL VS STATELESS FILTERING

STATEFUL (SECURITY GROUP)

Allows return traffic automatically.
Tracks connection state.

A → B
B → A (ALLOWED)

STATELESS (NACL / FIREWALL)

Does NOT remember connections.
Must allow both directions explicitly.

A → B (ALLOWED)
B → A (BLOCKED)

CHECK THE BASICS FIRST

1. CAN A PING B?
2. CAN B PING A?
3. CAN A TELNET/NC TO B:PORT?
4. CAN B TELNET/NC TO A:PORT?
5. CHECK SECURITY GROUPS
6. CHECK NACLs
7. CHECK ROUTE TABLES
8. CHECK FIREWALL RULES
9. CHECK RETURN PATH
10. CHECK ASYMMETRIC ROUTING

KEY COMPONENTS TO CHECK

COMPONENT	WHAT TO CHECK	COMMON ISSUE	HOW TO VERIFY
SECURITY GROUPS	Inbound rules on target Outbound rules on source	Missing inbound rule on target (If SG is too restrictive)	aws ec2 describe-security-groups Check both directions
NACLs	Inbound & outbound rules (Stateless)	Inbound allowed but outbound return blocked	aws ec2 describe-network-acls Ensure return traffic allowed
ROUTE TABLES	Routes for both directions (Source subnet & target subnet)	Missing or wrong return route	aws ec2 describe-route-tables Check next-hop & destination
FIREWALLS	Inbound & outbound policies Stateful or stateless?	Return traffic blocked by firewall	Check firewall logs Confirm rules for both directions
ASYMMETRIC ROUTING	Traffic takes different paths for request and reply	Return packet dropped due to asymmetric path	traceroute from both sides Check hop differences

USEFUL COMMANDS

1. TEST CONNECTIVITY

```
ping <IP>
nc -vz <IP> <PORT>
```

2. TRACE PATH

```
traceroute <IP>
mtr -rw <IP>
```

3. CAPTURE TRAFFIC

```
tcpdump -i eth0 host <IP>
and \{ port <PORT> }
```

4. CHECK LOCAL ROUTES

```
ip route show
route -n
```

5. CHECK CONNECTION STATE

```
ss -tulnp | grep <PORT>
conntrack -L | grep <IP>
```

NETWORK TROUBLESHOOTING METHODOLOGY

1 REPRODUCE

Confirm the issue from both sides. Define exactly what works and what fails.

2 BASIC CHECKS

Ping, port test, DNS, service listening, local firewall.

3 PATH ANALYSIS

Traceroute / MTR from both sides. Look for different paths.

4 ROUTING CHECK

Verify routes for both directions. Check default gateways.

5 FIREWALL / NACL

Verify inbound and outbound rules. Remember: NACLs are stateless.

6 DEEP INSPECTION

Check conntrack, tcpdump, logs. Look for drops or rejects.

7 RESOLVE & VERIFY

Fix the root cause. Test again from both directions. Document it.



SENIOR ENGINEER TIP

ONE-WAY CONNECTIVITY PROBLEMS ARE ALMOST ALWAYS ABOUT RETURN PATHS. ALWAYS THINK: "CAN THE REPLY PACKET FIND ITS WAY BACK?"



REAL WORLD LESSON

INFRASTRUCTURE IS A SYSTEM, NOT A SINGLE COMPONENT. A SINGLE MISCONFIGURED ROUTE, RULE, OR ACL CAN BREAK HALF OF THE CONVERSATION.

29. KUBERNETES HPA IS NOT SCALING DURING HEAVY TRAFFIC

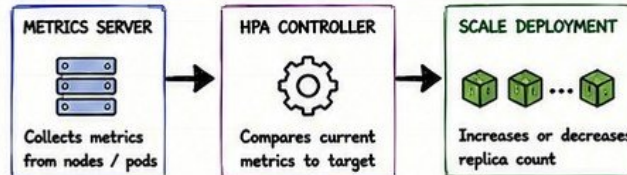
SCENARIO:

CPU USAGE APPEARS HIGH, BUT THE HORIZONTAL POD AUTOSCALER DOES NOT INCREASE REPLICAS.

INTERVIEW FOCUS

- ✓ METRICS SERVER
- ✓ RESOURCE REQUESTS
- ✓ HPA METRICS
- ✓ SCALING THRESHOLDS
- ✓ HPA STATUS
- ✓ STABILIZATION WINDOWS
- ✓ MAXIMUM REPLICAS
- ✓ CUSTOM METRICS
- ✓ APPLICATION BOTTLENECKS

HOW HPA WORKS (SIMPLIFIED)



HPA DECISION BASED ON:
METRICS + TARGET + MIN / MAX + STABILIZATION

COMMON REASONS HPA DOES NOT SCALE

- ✗ METRICS SERVER NOT WORKING
- ✗ MISSING CPU / MEMORY REQUESTS
- ✗ TARGET TOO HIGH
- ✗ HPA MAX REPLICAS REACHED
- ✗ STABILIZATION WINDOW BLOCKS SCALING
- ✗ CUSTOM METRICS MISCONFIGURED
- ✗ APPLICATION IS THE BOTTLENECK
- ✗ READINESS PROBES FAILING
- ✗ PODS ARE PENDING (NO CAPACITY)
- ✗ RESOURCE QUOTAS / LIMITRANGE

CHECK 1: METRICS SERVER

HPA RELIES ON METRICS SERVER FOR CPU / MEMORY. VERIFY IT FIRST.

```
kubectl get apiservice | grep metrics
kubectl get pods -n kube-system -l k8s-app=metrics-server
kubectl top nodes
kubectl top pods -n <namespace>
```

IF "kubectl top" COMMANDS FAIL,
HPA CANNOT GET METRICS!



CHECK 2: RESOURCE REQUESTS

HPA CANNOT CALCULATE CPU % WITHOUT CPU REQUESTS.

```
kubectl describe pod <pod-name> | grep -A 5 "Requests:"
kubectl get deploy <deploy-name> -o yaml |
grep -A 5 resources:
```

EVERY CONTAINER MUST HAVE
CPU REQUESTS SET!



CHECK 3: HPA STATUS

ALWAYS START HERE. IT GIVES THE REAL STORY.

```
kubectl get hpa
kubectl describe hpa <hpa-name>
```

LOOK FOR:

- ✓ CURRENT / TARGET VALUES
- ✓ DESIRED REPLICAS
- ✓ CURRENT REPLICAS
- ✓ EVENTS (WARNINGS / FAILURES)
- ✓ RECOMMENDATION REASON

HPA METRICS EXPLAINED

METRIC TYPE	DESCRIPTION	EXAMPLE
RESOURCE METRICS	CPU / MEMORY USAGE BASED ON REQUESTS	targetCPUUtilizationPercentage: 70
PODS METRICS	CUSTOM METRICS PER POD	Requests per second per pod
OBJECT METRICS	METRICS FOR K8S OBJECTS (DEPLOYMENT, NAMESPACE)	Queue length, RPS, etc.
EXTERNAL METRICS	METRICS OUTSIDE K8S (APIS, DB, QUEUES)	SQS queue length, API QPS

CHECK 4: SCALING CONFIGURATION

REVIEW MIN, MAX, TARGETS & WINDOWS.

```
kubectl get hpa <hpa-name> -o yaml
```

```
spec:
  minReplicas: 2                # lower bound
  maxReplicas: 10               # upper bound
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          - type: Utilization
            averageUtilization: 70  # target %
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 60 # wait before next scale up
    scaleDown:
      stabilizationWindowSeconds: 300 # wait before scale down
```

KEY POINTS

- TARGET TOO HIGH = HPA NEVER SCALES
- MAX REPLICAS REACHED = HPA STOPS SCALING
- STABILIZATION WINDOWS PREVENT FLAPPING
- SCALE UP CAN BE SLOW BY DESIGN



CHECK 5: CUSTOM METRICS (IF USING)

VERIFY METRICS ARE AVAILABLE.

```
kubectl get --raw /apis/custom.metrics.k8s.io/v1beta1
kubectl describe hpa <hpa-name>
```

LOOK FOR:

- ✓ METRIC VALUE
- ✓ TARGET VALUE
- ✓ HPA ABLE TO READ THE METRIC

CHECK 6: APPLICATION BOTTLENECKS

HPA INCREASES PODS BUT PERFORMANCE MAY NOT IMPROVE.

- DATABASE SLOW?
- EXTERNAL API LIMITS?
- DISK / IO SATURATION?
- THREAD POOL EXHAUSTED?
- MEMORY PRESSURE / GC?



DEBUGGING COMMANDS CHEAT SHEET

```
kubectl get hpa
kubectl describe hpa <hpa-name>
kubectl top pods -n <namespace>
kubectl get pods -n <namespace> -o wide
kubectl get deploy <deploy-name>
kubectl describe deploy <deploy-name>
kubectl get events -n <namespace> --sort-by=.lastTimestamp
kubectl logs <pod-name> --previous
```

SENIOR ENGINEER TIP



HPA IS NOT A MAGIC SOLUTION. IT ONLY REACTS TO METRICS. FIRST ENSURE METRICS ARE CORRECT, REQUESTS ARE SET, AND THE APPLICATION CAN ACTUALLY HANDLE MORE TRAFFIC.



REAL WORLD LESSON



IF HPA IS NOT SCALING, THE ISSUE IS ALMOST ALWAYS ONE OF THESE: METRICS, REQUESTS, CONFIGURATION, OR APPLICATION BOTTLENECK. FOLLOW THE CHECKLIST. DON'T GUESS—VERIFY.



REMEMBER

NO REQUESTS = NO METRICS
NO METRICS = NO SCALING
NO SCALING = UNHAPPY USERS

30. JENKINS CONTROLLER CRASHES DURING BUSINESS HOURS

SCENARIO:

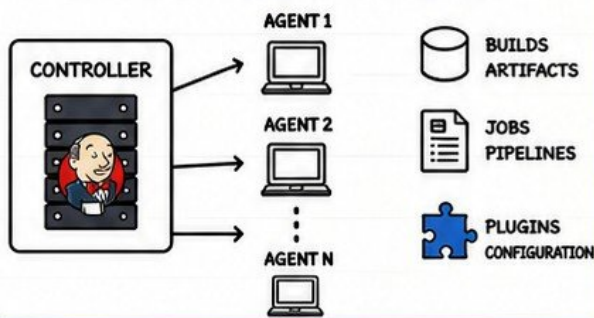
THE JENKINS CONTROLLER BECOMES UNAVAILABLE WHILE SEVERAL PRODUCTION DEPLOYMENTS ARE RUNNING.

INTERVIEW FOCUS

- ✓ CONTROLLER ARCHITECTURE
- ✓ AGENT WORKLOADS
- ✓ JVM MEMORY
- ✓ PLUGIN PROBLEMS
- ✓ BACKUP STRATEGY
- ✓ PIPELINE DURABILITY
- ✓ RECOVERY
- ✓ HIGH AVAILABILITY LIMITATIONS
- ✓ REDUCING CONTROLLER DEPENDENCY



JENKINS CONTROLLER ARCHITECTURE



IMPACT WHEN CONTROLLER CRASHES



- NEW PIPELINES CANNOT START
- RUNNING PIPELINES MAY FAIL
- JOB SCHEDULING STOPS
- WEB UI IS UNAVAILABLE
- API IS UNAVAILABLE
- BUILD HISTORY NOT ACCESSIBLE
- DEPLOYMENTS ARE BLOCKED

COMMON CAUSES

- HIGH JVM MEMORY CONSUMPTION / LEAK
- PLUGIN CONFLICT OR BUG
- DISK FULL OR I/O PRESSURE
- HEAVY BUILDS ON CONTROLLER
- CORE / JAVA / OS ISSUES
- SYSTEM UPDATES OR RESTARTS

HOW TO IDENTIFY THE PROBLEM

- 1 CHECK INFRASTRUCTURE: CPU, MEMORY, DISK, NETWORK.
- 2 REVIEW JENKINS LOGS:
`/var/jenkins_home/logs/jenkins.log`
- 3 CHECK JVM METRICS:
GC pauses, heap usage, thread count.
- 4 CHECK PLUGIN STATUS AND RECENT UPDATES.
- 5 VERIFY IF DISK IS FULL:
`df -h`
- 6 CHECK IF AGENTS ARE STILL ONLINE.

IMPORTANT COMMANDS

CHECK PROCESS
`ps -ef | grep java`

CHECK CPU & MEMORY
`top -p $(pgrep -f jenkins.war)`

CHECK DISK
`df -h`
`du -sh /var/jenkins_home/*`

CHECK JVM INFO (FROM JENKINS SERVER)
`jstat -gcutil $(pgrep -f jenkins.war) 5s`

CHECK AGENTS
`curl -s http://<agent-ip>:<port>/computer/api/json`

RESTART JENKINS (IF NEEDED)
`sudo systemctl restart jenkins`

RECOVERY STEPS

- 1 STABILIZE THE SERVER (CPU, MEMORY, DISK).
- 2 RESTART JENKINS SERVICE.
- 3 VERIFY JENKINS STARTS CORRECTLY.
- 4 CHECK PLUGINS FOR FAILURES.
- 5 VERIFY AGENTS COME BACK ONLINE.
- 6 RESUME QUEUED BUILDS.
- 7 COMMUNICATE STATUS TO THE TEAM.



PIPELINE DURABILITY

ENABLE DURABILITY TO SURVIVE CONTROLLER RESTARTS.
SET IN 'MANAGE JENKINS' → 'CONFIGURE SYSTEM'.

DURABILITY LEVELS

- LOW PERFORMANCE → RISK OF DATA LOSS
- MAX SURVIVABILITY → SAFEST (RECOMMENDED)
- BALANCED → GOOD COMPROMISE

ALSO USE:

- ✓ DISK PERSISTENCE (`/var/jenkins_home`)
- ✓ ATOMIC WRITES
- ✓ PIPELINE CHECKPOINTS

BACKUP STRATEGY

BACK UP `/var/jenkins_home` REGULARLY.



INCLUDE:

- ✓ JOBS
- ✓ CONFIGURATION
- ✓ PLUGINS
- ✓ CREDENTIALS
- ✓ ARTIFACT METADATA

EXAMPLE BACKUP COMMAND

`tar -czf jenkins_backup_$(date +%F).tar.gz /var/jenkins_home`

HIGH AVAILABILITY LIMITATIONS



- JENKINS CONTROLLER IS NOT NATIVE HA.
- TWO CONTROLLERS SHARING THE SAME `$JENKINS_HOME` CAN CORRUPT DATA.
- JENKINS REQUIRES MANUAL SETUP FOR HA (COMPLEX).
- PLUGINS MAY NOT SUPPORT HA.
- HA SOLUTIONS ADD OPERATIONAL OVERHEAD.

REALITY: MOST SETUPS RELY ON BACKUPS + FAST RECOVERY.

REDUCING CONTROLLER DEPENDENCY



- ✓ USE AGENTS FOR ALL BUILDS (KEEP CONTROLLER LIGHT).
- ✓ USE EXTERNAL TOOLS FOR ARTIFACT STORAGE (S3, NEXUS, JFROG).
- ✓ USE PIPELINE AS CODE (STORE IN GIT).
- ✓ USE SHARED LIBRARIES IN GIT.
- ✓ USE WEBHOOKS / EVENT TRIGGERS INSTEAD OF POLLING.
- ✓ MATURITY TIP: MOVE TO CI PLATFORMS WITH BETTER HA (GITHUB ACTIONS, GITLAB CI, CIRCLECI, TEKTON).

KEY TAKEAWAY



A JENKINS CONTROLLER OUTAGE STOPS DELIVERY.
KEEP IT LIGHT, MONITORED, BACKED UP, AND EASY TO RECOVER.



PAGE 31. A PRODUCTION DEPLOYMENT MUST BE ROLLED BACK IMMEDIATELY

⚠ SCENARIO

A RELEASE INTRODUCES CUSTOMER-FACING ERRORS FIVE MINUTES AFTER DEPLOYMENT.

INTERVIEW FOCUS

- ROLLBACK CRITERIA
- KUBERNETES ROLLOUT UNDO
- HELM ROLLBACK
- DATABASE COMPATIBILITY
- FEATURE FLAGS
- IMMUTABLE ARTIFACTS
- DEPLOYMENT VERIFICATION
- INCIDENT COMMUNICATION
- WHEN ROLLBACK ITSELF IS DANGEROUS



GOAL

RESTORE SERVICE SAFELY AND QUICKLY WITH MINIMAL RISK AND CLEAR COMMUNICATION.



IMMEDIATE ACTIONS (FIRST 2-5 MINUTES)

- 1 CONFIRM THE IMPACT
 - Check user reports, error rates, latency, and failed transactions.
- 2 ASSESS THE BLAST RADIUS
 - Which services, endpoints, or customers are affected?
- 3 STOP FURTHER AUTOMATION
 - Pause CI/CD pipelines if they auto-deploy.
- 4 COMMUNICATE EARLY
 - Notify team and stakeholders that an incident is in progress.

ROLLBACK CRITERIA

- ☒ User-facing errors or failed transactions
- ☒ High error rate (e.g., > 1%)
- ☒ Severe latency increase
- ☒ Dependency failures caused by the release
- ☒ Business or revenue impact
- ☒ SLO/SLA violations
- ☒ No quick mitigation available.

KUBERNETES ROLLOUT UNDO



- # Check rollout history
`kubectl rollout history deployment/<app>`
- # Rollback to previous revision
`kubectl rollout undo deployment/<app>`
- # Rollback to a specific revision
`kubectl rollout undo deployment/<app> --to-revision=<revision-number>`

HELM ROLLBACK



- # Check release history
`helm history <release-name>`
- # Rollback to previous revision
`helm rollback <release-name>`
- # Rollback to a specific revision
`helm rollback <release-name> <revision-number>`

DATABASE COMPATIBILITY



- Ensure schema changes are backward compatible.
- Avoid destructive migrations in the same deploy.
- Use expand-and-contract strategy.
- Verify application version matches DB version.
- Rollback may not fix data already changed by migrations.

FEATURE FLAGS



- If available, disable the new feature immediately.
- Feature flags can stop the blast radius without rollback.
- Prefer flags for high-risk features.

IMMUTABLE ARTIFACTS



- Always deploy immutable image versions.
- Never use `:latest` in production.
- Rollback is reliable only when artifacts are immutable and available.
- Store artifacts in a registry with retention.

DEPLOYMENT VERIFICATION



- After rollback, verify all health checks are green.
- Check error rates, latency, and user flows.
- Verify dependencies and background jobs.
- Confirm old version is serving all traffic.
- Monitor closely for at least 15-30 minutes.

INCIDENT COMMUNICATION



- Communicate fast, clearly, and often.
- Share: impact, action taken, current status, next update.
- Use public status page if available.
- Avoid technical jargon for customers.

⚠ WHEN ROLLBACK ITSELF IS DANGEROUS

- Database migrations are not backward compatible.
- Data has already been modified by the new version.
- The old version contains a critical security vulnerability.
- Rollback will cause data loss or inconsistency.
- Traffic has already been routed in a way that makes rollback unsafe.
- Always evaluate risk before rolling back.



SAFE ROLLBACK CHECKLIST

- ☐ Identify the issue and affected users
- ☐ Decide rollback vs mitigation vs hotfix
- ☐ Rollback using `kubectl` or `helm`
- ☐ Verify deployment and dependencies
- ☐ Monitor metrics and logs
- ☐ Communicate status and next steps
- ☐ Document the incident
- ☐ Plan a root cause analysis



PRODUCTION NOTE

The fastest rollback is not always the safest. Protect data, communicate clearly, and learn from every incident. 💡

PAGE 32. AWS BILL SUDDENLY DOUBLES



SCENARIO:

MONTHLY CLOUD SPENDING UNEXPECTEDLY INCREASES BY ALMOST **100%**.



INTERVIEW FOCUS

- COST EXPLORER
- RESOURCE TAGS
- DATA TRANSFER
- NAT GATEWAY
- UNUSED RESOURCES
- EBS
- SNAPSHOTS
- LOGGING
- AUTO SCALING
- KUBERNETES RESOURCE GROWTH
- COST ANOMALY INVESTIGATION

1. START WITH AWS COST EXPLORER



- Open Cost Explorer
- Set time range: Last 30/60/90 days
- Group by: Service → Linked Account → Region
- Look for the biggest jump
- Compare with previous period
- Use "Daily" granularity to find the spike day



2. CHECK COST ANOMALY DETECTION



- Go to Billing > Cost Anomaly Detection
- Review any detected anomalies
- Check which services and accounts were flagged
- Validate if the spike is expected or truly abnormal

3. INVESTIGATE COMMON COST DRIVERS

DATA TRANSFER



- High egress out of AWS
- Cross-AZ traffic
- Cross-region replication
- CloudFront misconfig
- S3 Transfer Acceleration

NAT GATEWAY



- NAT Gateway charges are per GB processed
- Large private subnet traffic can be expensive
- Check for unnecessary traffic via NAT

UNUSED RESOURCES



- Stopped but allocated
- EC2 instances
- Unused Load Balancers
- Unattached Elastic IPs
- Idle RDS instances
- Old Elastic IPs

EBS VOLUMES



- Large volumes provisioned but unused
- High IOPS or throughput settings
- Old volumes not deleted after workload removal

SNAPSHOTS



- Manual snapshots accumulating
- Automated snapshots with long retention
- Cross-region snapshots

LOGGING



- CloudWatch Logs ingestion volume
- High retention periods
- Verbose application logs
- VPC Flow Logs volume

AUTO SCALING



- Too aggressive scaling policies
- Instances not scaling in properly
- Minimum capacity too high

KUBERNETES RESOURCE GROWTH



- More pods running
- Increased node count
- High persistent volumes
- LoadBalancer services creating AWS resources

RESERVED CAPACITY MISSING



- On-Demand usage instead of Reserved Instances
- No Savings Plans
- Heavy usage without commitment

OTHER POSSIBLE CAUSES



- Third-party tools
- Large AWS Marketplace charges
- Data processing services (EMR, Athena, Glue)

4. USE COST EXPLORER FILTERS



- Filter by: Service, Region, Linked Account, Usage Type, Tag, Operation
- Drill down from top-level service to individual resources
- Identify the exact resource(s) driving the cost

5. CHECK RESOURCE TAGS



- Go to Cost Explorer → Group by: Tag
- Missing or inconsistent tags make it hard to track costs
- Enforce tagging policies
- Use cost allocation tags for business ownership

6. DEEP DIVE WITH CUR



- Enable Cost and Usage Report (CUR)
- Analyze with Athena, QuickSight, or Redshift
- Get detailed line-items (usage type, resource ID, operation, etc.)
- Best for advanced cost forensics

ACTION PLAN



1. Confirm the spike period and service
2. Identify the top cost driver
3. Investigate the root cause
4. Stop or right-size unnecessary usage
5. Add missing tags
6. Implement cost controls and budgets
7. Monitor and prevent recurrence

8. COST CONTROL BEST PRACTICES



- Use Budgets with alerts
- Enable Cost Anomaly Detection
- Rightsize resources regularly
- Use Reserved Instances / Savings Plans
- Set lifecycle rules for logs & snapshots
- Review Auto Scaling policies
- Enforce tagging and cost allocation

PRODUCTION NOTE

A cost spike is usually a symptom, not the root cause. Always combine data (Cost Explorer, CUR, tags) with context (deployments, traffic, business events). Good engineers investigate, not just react.



VERIQTA

PAGE 33. APPLICATION LOGS DISAPPEAR DURING AN INCIDENT



SCENARIO:

A POD CRASHES, AND ENGINEERS DISCOVER THE LOGS NEEDED FOR TROUBLESHOOTING ARE NO LONGER AVAILABLE.



INTERVIEW FOCUS

- EPHEMERAL CONTAINERS
- LOG RETENTION
- STDOUT AND STDERR
- PREVIOUS CONTAINER LOGS
- CENTRALIZED LOGGING
- STRUCTURED LOGGING
- FLUENT BIT
- CORRELATION IDS
- ELASTICSEARCH OR OPENSEARCH

1. WHY LOGS DISAPPEAR

- POD CRASHES AND IS RESTARTED
- CONTAINER FILE SYSTEM IS EPHEMERAL
- LOGS WRITTEN TO LOCAL FILES ARE LOST
- PREVIOUS CONTAINER IS REMOVED BY KUBELET
- NODES MAY BE DRAINED OR TERMINATED

POD CRASHES



POD RESTARTED



OLD CONTAINER REMOVED



LOGS GONE!

2. IMMEDIATE INVESTIGATION STEPS

- 1 Check pod status and events
`kubectl describe pod <pod-name>`
- 2 Get previous container logs (if available)
`kubectl logs <pod-name> --previous`
- 3 Check node logs (kubelet)
`journalctl -u kubelet -f`
- 4 Use ephemeral container for live debugging
`kubectl debug -it <pod-name> --image=busybox`



3. EPHEMERAL CONTAINERS (LIVE DEBUGGING)

- Inject a temporary container into a running or crashed pod
- Share namespaces with target container
- Do not restart the pod
- Great for checking files, processes, and runtime state

```
kubectl debug -it <pod-name> \
--image=busybox \
--target=<container-name> \
--share-processes --share-network --share-ipc
```

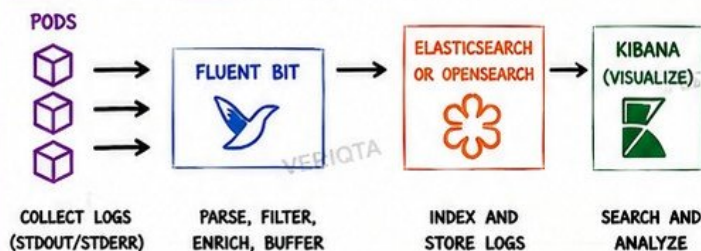


4. STDOUT AND STDERR (THE GOLD STANDARD)

- Write logs to stdout and stderr
- Captured by the container runtime
- Automatically collected by logging agents
- Survives container restarts (if centralized)
- Avoid writing logs only to files inside the container



5. CENTRALIZED LOGGING ARCHITECTURE



6. FLUENT BIT (LIGHTWEIGHT LOG COLLECTOR)

- Runs as DaemonSet on each node
- Reads container stdout/stderr
- Parses and enriches logs
- Forwards to Elasticsearch/OpenSearch
- Low CPU and memory usage

SAMPLE FLUENT BIT INPUT (TAIL CONTAINER LOGS)

```
[INPUT]
Name      tail
Path      /var/log/containers/*.log
Parser    docker
Tag        kube.*
Refresh_Interval 5
Mem_Buf_Limit 5MB
```

7. LOG RETENTION



- Define retention based on compliance and troubleshooting needs
- Example: 30, 90, or 180 days
- Use ILM (Index Lifecycle Management) in Elasticsearch/OpenSearch
- Monitor storage growth

8. PREVIOUS CONTAINER LOGS



- Use `--previous` flag before the container restarts
- Works only if the container has not been garbage collected

```
kubectl logs <pod-name> --previous
```

9. STRUCTURED LOGGING



- Log in JSON format
- Easier to search and filter
- Consistent fields
- Supports dashboards
- Improves alerting and correlation

10. CORRELATION IDS



- Add a unique ID to every request
- Pass it across services
- Include in every log line
- Essential for tracing problems across systems

PRODUCTION NOTE

IF LOGS LIVE ONLY INSIDE CONTAINERS, YOU WILL LOSE THEM. CENTRALIZE EARLY. RETAIN WISELY. LOG STRUCTURED. TRACE EVERYTHING.



COMMON MISTAKES

- Writing logs only to files
- Insufficient log context
- No log retention policy
- Overly verbose logs
- No centralization
- Not monitoring logging pipeline
- No correlation IDs
- Ignoring logging costs

BEST PRACTICES



- Log to stdout/stderr
- Centralize with Fluent Bit
- Use structured logging
- Retain logs appropriately
- Monitor logging pipeline
- Use correlation IDs
- Review logs during incidents
- Test log retrieval regularly

VERIQTA



YouTube
/veriqta



GitHub
/veriqta



LinkedIn
/veriqta



X (Twitter)
@veriqta



Instagram
@veriqta



Telegram
/veriqta

PAGE 34. PRODUCTION API IS HEALTHY, BUT USERS REPORT HIGH LATENCY



SCENARIO:

HEALTH CHECKS REMAIN GREEN WHILE CUSTOMERS REPORT THAT REQUESTS TAKE SEVERAL SECONDS.



INTERVIEW FOCUS

- WHY HEALTH CHECKS ARE INSUFFICIENT
- PERCENTILES
- P95 AND P99 LATENCY
- DISTRIBUTED TRACING
- DEPENDENCY LATENCY
- DATABASE QUERIES
- CACHE MISSES
- THREAD POOLS
- SATURATION
- RED METRICS

1. WHY HEALTH CHECKS ARE INSUFFICIENT



Health Check Endpoint

Returns 200 OK (Fast & Lightweight)

HEALTHY



Real User Request (Complex Workflow)

Slow Dependencies, DB, Cache, External APIs

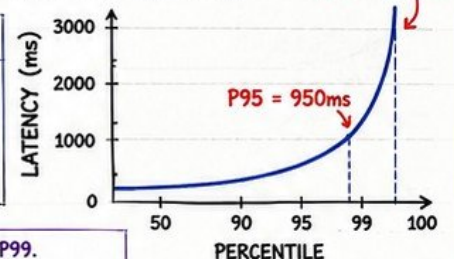
HIGH LATENCY

Health checks only verify that the service is UP, not that it is performing well under real load.

2. PERCENTILES MATTER

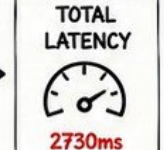
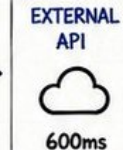
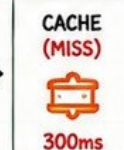
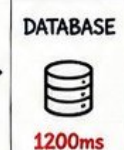
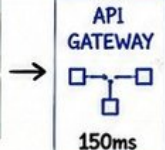
AVERAGE CAN LIE. PERCENTILES TELL THE TRUTH.

METRIC	VALUE (ms)
AVERAGE	120
P50 (MEDIAN)	80
P95	950
P99	2500



FOCUS ON P95 AND P99. THE TAIL IS WHERE YOUR USERS SUFFER.

3. FOLLOW THE REQUEST: END-TO-END LATENCY BREAKDOWN



IDENTIFY SLOW COMPONENTS, NOT JUST THE SYMPTOM.

4. DISTRIBUTED TRACING



- Trace every request across services
- Identify slow spans quickly
- Use Trace ID to follow a single request
- Great tools: Jaeger, Zipkin, AWS X-Ray, Datadog, New Relic

5. DEPENDENCY LATENCY



- External services degrade
- APIs timeout or throttle
- DNS, network, SSL handshake
- Monitor and set sane timeouts
- Fail fast, don't wait

6. DATABASE QUERIES



- Slow queries kill performance
- Check execution plans
- Look for missing indexes
- N+1 queries are killers
- Connection pool saturation

7. CACHE MISSES



- High miss rate = more DB load
- Check TTL, eviction, key design
- Warm up cache after deploys
- Monitor hit rate (%)

8. THREAD POOLS



- Too many threads → context switching
- Too few threads → queueing
- Monitor active, queued, completed
- Tune pool size based on load

9. SATURATION



- CPU near 100% = slow everything
- Memory pressure causes GC spikes
- Disk or network bottlenecks
- Saturation creates high tail latency

10. RED METRICS (THE GOLD STANDARD)

R	Rate	Requests per second
E	Errors	Error rate (%)
D	Duration	Latency (P50, P95, P99)

EXAMPLE DASHBOARD



SENIOR ENGINEER TIP

- If users are slow but health checks are green, it's not an availability problem. It's a performance problem.
- Stop looking at averages. Look at the tail.
- Always follow one request end-to-end. Metrics tell you WHAT. Traces tell you WHY.

PAGE 35. AN ENGINEER ACCIDENTALLY, DELETES A KUBERNETES PRODUCTION RESOURCE



SCENARIO:

A CRITICAL KUBERNETES OBJECT IS MANUALLY DELETED FROM THE PRODUCTION CLUSTER.



INTERVIEW FOCUS

- IMMEDIATE IMPACT ASSESSMENT
- GITOPS RECOVERY
- DECLARATIVE CONFIGURATION
- OWNERREFERENCES
- PERSISTENT DATA
- RBAC
- AUDIT LOGS
- ADMISSION CONTROLS
- BACKUP AND RESTORE
- PREVENTING RECURRENCE

1. IMMEDIATE IMPACT ASSESSMENT



- What was deleted? (kind, name, namespace)
- What depends on it?
- Is the workload still running?
- Is any data at risk?
- Check events, logs, and alerts.

USEFUL COMMANDS

```
kubectl get events --sort-by=.metadata.creationTimestamp
kubectl get <resource-kind> -n <namespace>
kubectl describe <resource-kind> <name> -n <namespace>
```

2. GITOPS RECOVERY



- If using GitOps (Argo CD, Flux), the controller will detect drift and re-create the object.
- Ensure the controller is healthy.
- Force sync if needed.
- Verify the object is restored and healthy.

ARGO CD EXAMPLE

```
argocd app get <app-name>
argocd app sync <app-name> --force
argocd app wait <app-name> --health
```

3. DECLARATIVE CONFIGURATION



- Kubernetes is declarative: the desired state is in manifests.
- Ensure the manifest exists in Git.
- Commit missing changes if needed.
- Avoid "kubectl edit" in production.

CHECK MANIFEST IN GIT

```
git log -- <path/to/manifest.yaml>
git show HEAD:<path/to/manifest.yaml>
```

4. OWNERREFERENCES



- Objects with OwnerReferences are automatically recreated when the owner exists.
- Ensure controllers (Deployments, StatefulSets, Jobs, etc.) are healthy.
- Avoid deleting owners by mistake.

5. PERSISTENT DATA



- Check if PVCs or PVs were deleted.
- If PVC was deleted but PV has Retain policy, data may still exist.
- If data is lost, recovery becomes much harder.
- Verify StatefulSet volume claims.

CHECK PV STATUS

```
kubectl get pv,pvc -A
```

6. RBAC (LEAST PRIVILEGE)



- Limit who can delete critical resources.
- Use least privilege roles.
- Use separate admin and operational roles.
- Regularly review ClusterRoles and bindings.

7. AUDIT LOGS



- Identify who deleted the resource.
- When did it happen?
- From which IP or user?
- What was the exact command?

VIEW AUDIT LOGS

```
kubectl logs -n kube-system -l component=kube-apiserver --since=2h | grep "<resource-name>"
```

8. ADMISSION CONTROLS



- Use Admission Controllers (OPA Gatekeeper, Kyverno) to block dangerous actions.
- Prevent deletion of critical resources in production.
- Enforce policies via code.

9. BACKUP AND RESTORE



- Backup etcd regularly.
- Validate backups often.
- For cluster-level disasters, restore from backup.
- Tools: Velero, etcdctl snapshots.

BACKUP EXAMPLE (VELERO)

```
velero backup create prod-backup --include-namespaces "*" --wait
```

10. PREVENTING RECURRENCE



TRAIN TEAM ON KUBERNETES BEST PRACTICES



USE GITOPS AS THE SOURCE OF TRUTH



ENFORCE RBAC AND LEAST PRIVILEGE



APPLY ADMISSION POLICIES



MONITOR AUDIT LOGS REGULARLY



TEST BACKUPS AND RESTORES



IMPLEMENT ALERTS FOR CRITICAL DELETIONS



KEY TAKEAWAY

Kubernetes is resilient, but human error can still cause major incidents. Strong processes, GitOps, observability, and guardrails prevent small mistakes from becoming outages.

REMEMBER

- ✓ Declarative state + GitOps = Faster recovery
- ✓ Logs + Audit = Accountability
- ✓ Least privilege + Policies = Prevention
- ✓ Backups = Last line of defense

PRODUCTION NOTE

Practice recovery in staging. Know your restore procedure before you need it.

36. TERRAFORM APPLY FAILS HALFWAY THROUGH

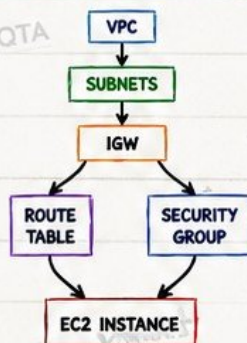
SCENARIO

Terraform successfully creates several resources and then fails before completing the deployment.

Terraform

WHAT HAPPENS DURING APPLY?

- 1 Terraform builds a dependency graph.
- 2 It creates/updates resources in order.
- 3 It writes successful resources to state.
- 4 If an error occurs, apply stops immediately.
- 5 Resources created before the failure remain in your infrastructure and state.



INTERVIEW FOCUS

- ✓ Terraform's partial execution behavior
- ✓ State after failure
- ✓ Refresh and plan
- ✓ Dependency graph
- ✓ Idempotency
- ✓ Manual changes
- ✓ Import
- ✓ Safe retry
- ✓ Why blindly rerunning can be dangerous

EXAMPLE FAILURE FLOW

- 1 VPC ✓ Created
- 2 Subnets ✓ Created
- 3 Internet Gateway ✓ Created
- 4 Route Table ✓ Created
- 5 Security Group ✓ Created
- 6 EC2 Instance ✗ Failed (Insufficient IAM permission)

RESULT: Some resources exist, some do not. State is partially updated.

STATE AFTER FAILURE

- Resources created before the failure are recorded in the state file.
- Resources not created are not in the state.
- Terraform does NOT automatically roll back what was already created.

terraform.tfstate (Partial State)

- ✓ VPC
- ✓ Subnets
- ✓ IGW
- ✓ Route Table
- ✓ Security Group
- ✗ EC2 Instance

⚠ DANGER: DO NOT BLINDLY RE-RUN APPLY

- You may recreate resources that already exist.
- You may hit naming conflicts.
- You may create duplicates (e.g., Security Groups, Load Balancers, DNS records).
- You may violate quotas.
- You may cause downtime.
- You may overwrite manual hotfixes.



RECOVER SAFELY (STEP-BY-STEP)

- 1 Check the error message carefully.
- 2 Identify which resource failed.
- 3 Run: `terraform state list`
→ See what Terraform thinks exists.
- 4 Run: `terraform plan`
→ Understand the real drift.
- 5 If something was manually created or changed, decide:
 - Keep it → Import it
 - Remove it → Destroy it
- 6 Fix the root cause.
- 7 Re-run `terraform apply`.

KEY COMMANDS

`terraform plan`
- See what will be created/changed.

`terraform state list`
- List resources in state.

`terraform state show <address>`
- Show details of a resource in state.

`terraform refresh`
- Sync state with real infrastructure.

`terraform import <address> <id>`
- Import existing resource into state.

`terraform apply`
- Apply changes safely.

WHEN TO IMPORT?

IMPORT WHEN:

- A resource was created manually.
- A resource was created outside Terraform.
- You need Terraform to manage it now.

IMPORT EXAMPLE

`terraform import aws_s3_bucket.logs my-prod-logs`
Now it is tracked in state.

PRODUCTION TIP

Always review plans. Understand the blast radius. Never treat apply like a button. Terraform is powerful, but not forgiving when used carelessly.

REAL WORLD LESSON

Good engineers don't just run commands. They understand state, dependencies, drift, and the impact of every action.



Think. Verify. Plan. Apply.
That is real DevOps.



Terraform is declarative, but production is not. Be intentional. Be safe. Be accountable.

38. A KUBERNETES CLUSTER IS HEALTHY, BUT ONE AVAILABILITY ZONE FAILS

SCENARIO

An entire Availability Zone (AZ) becomes unavailable due to a network outage. The Kubernetes control plane is healthy, but all nodes and resources in that AZ are down.

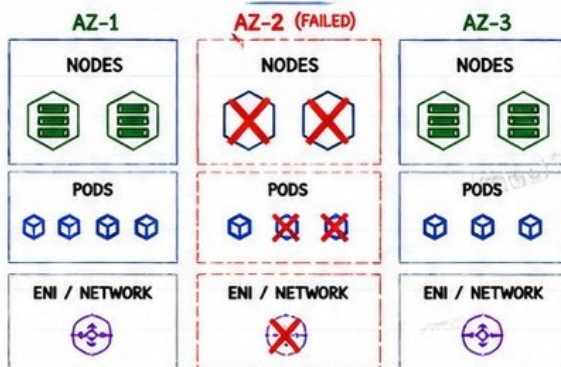


CLUSTER CONTROL PLANE: HEALTHY ✓

INTERVIEW FOCUS

- ✓ Multi-AZ node groups
- ✓ Pod distribution
- ✓ Topology spread constraints
- ✓ PodDisruptionBudgets (PDBs)
- ✓ Persistent volumes
- ✓ Stateful workloads
- ✓ Load balancers
- ✓ Capacity in surviving zones
- ✓ Failure-domain design

MULTI-AZ CLUSTER ARCHITECTURE (EXAMPLE)



When AZ-2 fails, pods on those nodes are evicted. Workloads must continue running in AZ-1 and AZ-3.

POD DISTRIBUTION & TOPOLOGY SPREAD CONSTRAINTS

WITHOUT TOPOLOGY SPREAD

Too many pods land in the same AZ. High blast radius.



WITH TOPOLOGY SPREAD

Pods are evenly distributed. Lower blast radius.



EXAMPLE: TOPOLOGY SPREAD CONSTRAINT

```
topologySpreadConstraints:
- maxSkew: 1
  topologyKey: topology.kubernetes.io/zone
  whenUnsatisfiable: DoNotSchedule
- labelSelector:
    matchLabels:
      app: my-app
```

EFFECT:

Kubernetes ensures pods are balanced across availability zones. If one zone fails, only 1 zone worth of pods are affected. ✓

POD DISRUPTION BUDGETS (PDBs)

Protects against voluntary disruptions during a failure.

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: my-app-pdb
spec:
  minAvailable: 2 # or maxUnavailable: 1
  selector:
    matchLabels:
      app: my-app
```

EFFECT:

Ensures Kubernetes does not evict more pods than allowed, keeping the app available during failures or upgrades. ✓

STATEFUL WORKLOADS & PERSISTENT VOLUMES

BEST PRACTICES

- ✓ Use Multi-AZ Storage Classes (gp3/io2 or EBS Multi-Attach for some use cases).
- ✓ Avoid single-AZ EBS volumes for critical workloads.
- ✓ Use StatefulSets with Pod Anti-Affinity.
- ✓ Test failover and recovery.

RISK IF NOT DESIGNED WELL

- ✗ PV in failed AZ = pod Pending until volume is reattached.
- ✗ Stateful workloads may become unavailable.
- ✗ Data loss risk if replicas are not across zones.



LOAD BALANCERS

- Use AWS ALB/NLB across multiple AZs.
- If one AZ fails, traffic is routed to healthy AZs.
- Ensure targets exist in at least 2 healthy AZs.



CAPACITY IN SURVIVING ZONES

Surviving AZs must have enough:

- CPU
- Memory
- ENIs / IPs
- Storage IOPS / Throughput

Cluster Autoscaler should be able to scale in remaining AZs.



FAILURE-DOMAIN DESIGN PRINCIPLES

- ① Assume a single AZ will fail.
- ② Distribute compute, storage, and networking.
- ③ Isolate stateful components.
- ④ Design for graceful degradation.
- ⑤ Continuously test failover.



KEY TAKEAWAY

A healthy cluster is not enough. Your application must survive the loss of an entire AZ. Design for failure, not for the best day.



“High availability is not a feature. It is an architecture decision. Design for failure before failure designs for you.”

REAL WORLD LESSON

Outages don't happen at the Kubernetes level. They happen at the infrastructure and design level. Plan your failure domains. Protect your users.



39. PRODUCTION IS DOWN, BUT EVERY DASHBOARD IS GREEN

SCENARIO

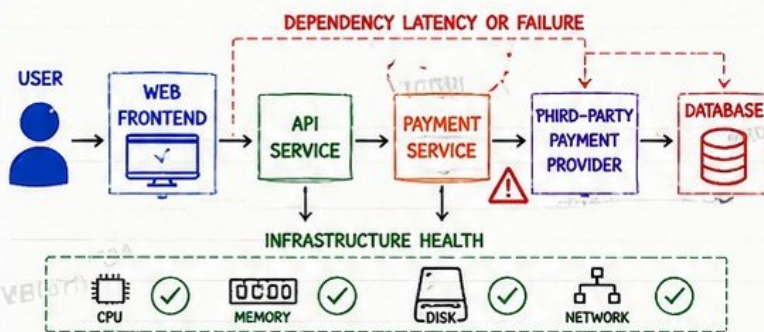
Customers cannot complete transactions, yet CPU, memory, disk, and infrastructure dashboards appear normal.



INTERVIEW FOCUS

- ✓ Infrastructure metrics vs user experience
- ✓ SLI and SLO design
- ✓ Synthetic monitoring
- ✓ Black-box monitoring
- ✓ Business metrics
- ✓ Application metrics
- ✓ Distributed tracing
- ✓ Dependency failures
- ✓ Alert design
- ✓ Observability gaps

WHY DASHBOARDS ARE GREEN BUT USERS ARE IMPACTED



Infrastructure looks healthy, but the dependency is slow/failing.
Users experience errors, timeouts, or long delays.

INFRASTRUCTURE METRICS vs USER EXPERIENCE

INFRASTRUCTURE METRICS (What servers report)

- ✓ CPU: Normal
- ✓ Memory: Normal
- ✓ Disk I/O: Normal
- ✓ Network: Normal
- ✓ Pod Status: Running

USER EXPERIENCE (What users feel)

- ✗ Slow Checkout
- ✗ Payment Failures
- ✗ Gateway Timeouts
- ✗ Errors
- ✗ Abandonment

≠

A healthy server does not mean a healthy service.

OBSERVABILITY PILLARS YOU NEED

1. SYNTHETIC MONITORING



- Simulate real user journeys.
- Detect issues from outside.

2. BLACK-BOX MONITORING



- Check endpoints like a user.
- Validate availability and response time.

3. APPLICATION METRICS



- Request rate
- Error rate
- Latency (p95/p99)
- Saturation

4. DISTRIBUTED TRACING



- See request flow across services.
- Find the slow or failing dependency.

5. BUSINESS METRICS



- Orders completed
- Transactions success
- Revenue
- Conversion rate

DEPENDENCY FAILURES HIDE HERE

- External API slowness or outages
- DNS resolution issues
- Payment gateway latency
- Database lock contention
- Message queue backlogs
- Cache stampede
- Third-party rate limits



Users blame you, but the problem is somewhere else.

SLI AND SLO DESIGN (EXAMPLE)

SERVICE	SLI (WHAT YOU MEASURE)	SLO (TARGET)	ERROR BUDGET
Checkout API	Successful checkout rate	99.9%	0.1%
Payment Service	Payment success rate	99.5%	0.5%
Web Frontend	Page load time (p95)	< 2 seconds	N/A
User Journey	End-to-end success rate	99%	1%



SLIs tell you WHAT to measure.
SLOs tell you HOW GOOD is good enough.

ALERT DESIGN THAT WORKS

Alert on symptoms that impact users:

- ✓ High error rate
- ✓ High latency (p95/p99)
- ✓ Low success rate
- ✓ SLO burn rate

Avoid alerts on:

- ✗ CPU high but no user impact
- ✗ Memory high but no user impact
- ✗ Infrastructure alone

COMMON OBSERVABILITY GAPS

- No synthetic monitoring
- No black-box monitoring
- No business metrics
- No distributed tracing
- Missing dependency visibility
- Alerts only on infra metrics
- No SLI/SLO definition
- No correlation across layers

If you can't see it, you can't fix it.



REAL WORLD LESSON

A healthy server does not mean a healthy service.

Users don't care about your CPU. They care about outcomes.
Observe what matters. Alert on what hurts. Design for the user experience.

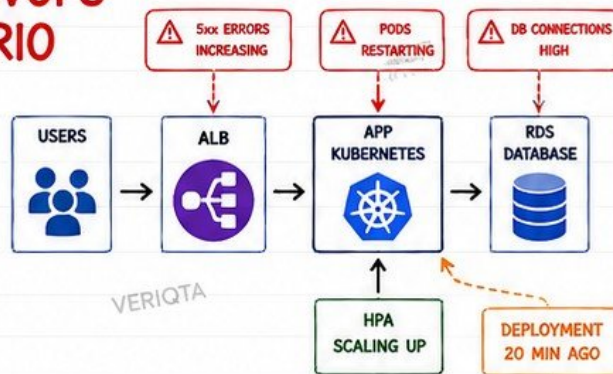
WHAT A SENIOR ENGINEER DOES

- ✓ Correlate user impact with telemetry.
- ✓ Trace the request end-to-end.
- ✓ Identify the failing dependency.
- ✓ Mitigate or degrade gracefully.
- ✓ Communicate clearly.
- ✓ Improve observability to prevent repeat.

40. FINAL SENIOR DEVOPS INCIDENT SCENARIO

SCENARIO

At 2:15 AM, alerts show rising API latency.
Kubernetes pods begin restarting.
ALB 5xx errors increase.
RDS connections approach their limit.
HPA adds pods, but the problem becomes worse.
A deployment occurred 20 minutes earlier.



CANDIDATE MUST DEMONSTRATE:

- ✓ Incident triage
- ✓ Blast-radius assessment
- ✓ Timeline construction
- ✓ Change correlation
- ✓ Kubernetes investigation
- ✓ AWS investigation
- ✓ Database investigation
- ✓ Logs, metrics, and traces
- ✓ Rollback decision
- ✓ Traffic mitigation
- ✓ Communication
- ✓ Evidence preservation
- ✓ Root-cause reasoning
- ✓ Recovery verification
- ✓ Post-incident actions



THE INTERVIEWER ASKS:

"You are the on-call DevOps engineer. What do you do?"

1. IMMEDIATE TRIAGE (FIRST 5 MINUTES)

- ✓ Acknowledge the alert.
- ✓ Follow the runbook.
- ✓ Assess user impact and severity.
- ✓ Identify the most affected service.
- ✓ Check recent changes (deployments, configs).
- ✓ Create an incident timeline.
- ✓ Notify the team and stakeholders.

TRIAGE COMMANDS

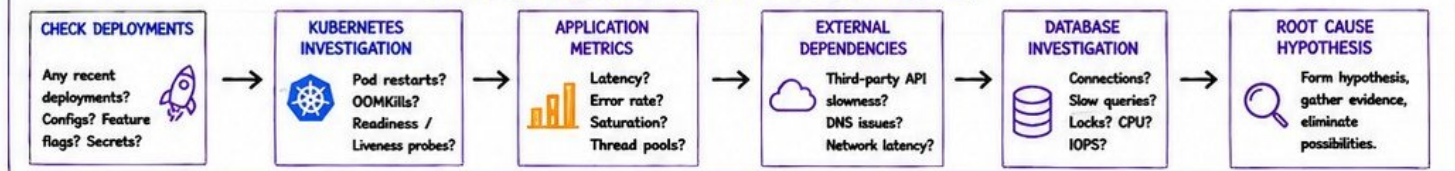
```

kubectl get pods -A --sort-by=.status.startTime
kubectl get hpa -A
kubectl describe hpa <name>
kubectl get events -A --sort-by=.lastTimestamp
aws elbv2 describe-target-health --target-group-arn <tg-arn>
aws cloudwatch get-metric-statistics --namespace AWS/ApplicationELB
aws rds describe-db-instances --db-instance-identifier <id>
kubectl rollout history deploy/<app> -n <namespace>
kubectl rollout status deploy/<app> -n <namespace>
  
```

2. BLAST RADIUS ASSESSMENT

- Which users are affected?
- Which regions / AZs are impacted?
- Which services depend on the failing one?
- Is this a partial or full outage?
- Check error rates, latency, and saturation.
- Identify downstream dependencies at risk.
- Decide what to protect first.

3. INVESTIGATION FLOW (NARROW THE SEARCH SPACE)



KEY INVESTIGATIONS

KUBERNETES

- kubectl logs, describe, events
- Check pod restarts & probes
- Node pressure / capacity
- Resource limits & requests

AWS

- ALB target health & 5xx
- CloudWatch metrics & alarms
- RDS connections & performance
- ECS / EC2 / ASG healthy?

DATABASE

- Active connections
- Slow query log
- Locks / waits
- CPU, Memory, IOPS

MITIGATION ACTIONS



TRAFFIC MITIGATION

Throttle traffic, enable rate limiting, block abusive clients if needed.



APPLICATION PROTECTION

Enable circuit breakers, increase timeouts carefully, shed non-critical traffic.



CAPACITY PROTECTION

Stop unnecessary autoscaling. Protect database connections. Queue or buffer when possible.

ROLLBACK DECISION

- Is the recent deployment the likely cause?
- Does rollback reduce risk?
- Are database changes backward compatible?
- Execute rollback safely.
- Verify previous version stability.



7. COMMUNICATION

- Communicate early and frequently.
- Share impact, status, and ETA.
- Use incident channel.
- Escalate when needed.
- Keep leadership informed.



EVIDENCE PRESERVATION

- Save logs, metrics, and traces.
- Capture screenshots of dashboards.
- Record timestamps of all actions.
- Preserve configs, events, and changes.



9. RECOVERY VERIFICATION

- Error rate back to normal?
- Latency within SLO?
- Database connections stable?
- Pods stable and healthy?
- Synthetic checks passing?
- User transactions successful?



10. POST-INCIDENT ACTIONS

- Write a clear post-mortem.
- Identify root cause and contributing factors.
- Document what worked and what failed.
- Create action items and follow through.
- Update runbooks and alerts.
- Share learnings across the team.



FINAL INTERVIEW TEST

The strongest candidate should not jump randomly between commands. They should:

- ✓ Form hypotheses
- ✓ Protect production
- ✓ Gather evidence
- ✓ Restore service
- ✓ Reduce the search space
- ✓ Investigate the deeper root cause